



北京邮电大学

Beijing University of Posts and Telecommunications

神经网络

戚 琦

网络与交换技术国家重点实验室 网络智能研究中心 科研楼511

qiqi8266@bupt.edu.cn

13466759972

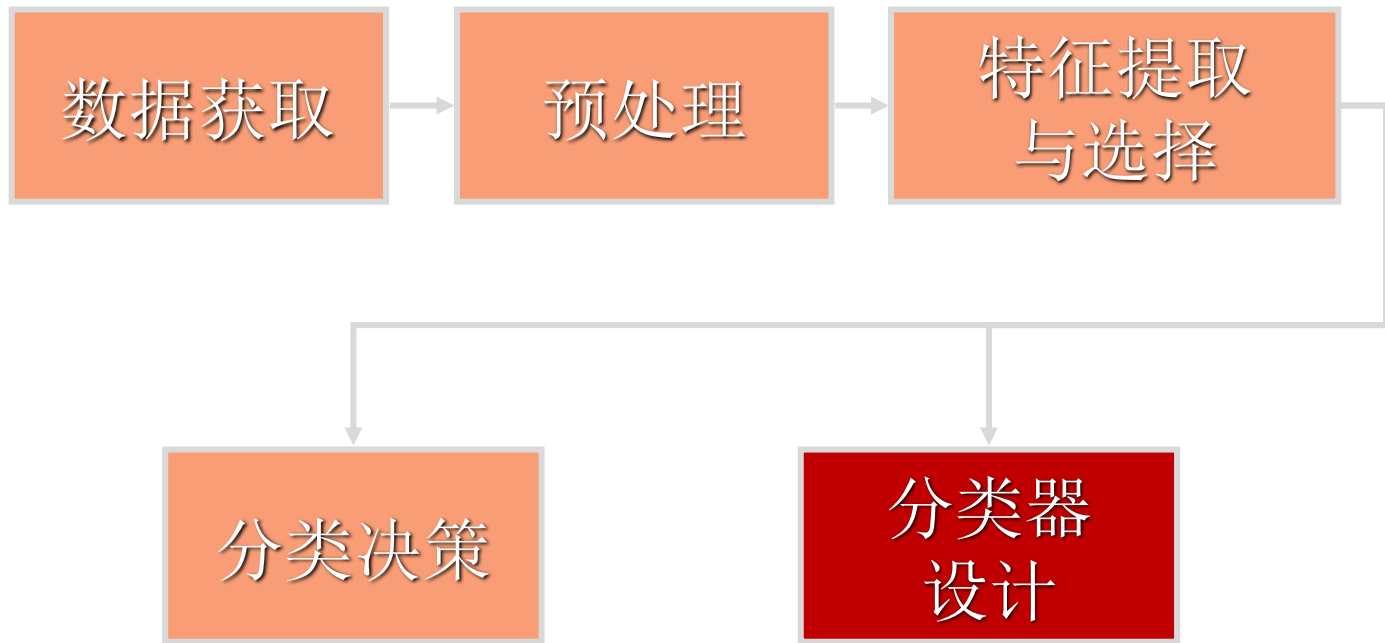


- 神经网络概述
- 神经元结构
- 多层神经网络
- 激活函数
- 梯度下降
- 反向传播算法
- 神经网络训练方法

周志华, 《机器学习》
浙大胡老师《机器学习》视频课
台大李宏毅老师《Machine Learning》视频课



回顾：监督学习



人工神经网络 (Artificial Neural Networks-----ANN)

- **生物神经网络** (Natural Neural Network, NNN): 由中枢神经系统 (脑和脊髓) 及周围神经系统 (感觉神经、运动神经等) 所构成的错综复杂的神经网络, 其中最重要的是**脑神经系统**。
 - **人工神经网络** (Artificial Neural Networks, ANN): **模拟人脑神经网络**的结构和功能, 运用大量简单处理单元经广泛连接而组成的人工网络系统。
- ✓ T. Koholen的定义: “人工神经网络是由具有适应性的简单单元组成的广泛并行互连的网络, 它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应。”

■ 探索时期：（开始于20世纪40年代）

- 1943年，**麦克劳（W. S. McCulloch）和匹茨（W. A. Pitts）**首次提出一个神经网络模型——**M—P模型**。
- 1949年，**赫布（D. O. Hebb）**提出改变神经元连接强度的Hebb学习规则。

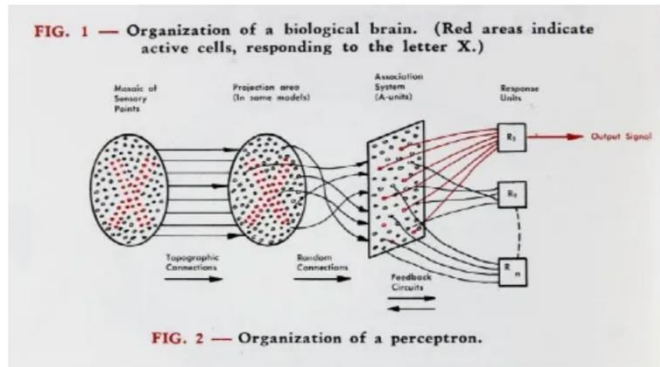
■ 第一次热潮时期：（20世纪50年代末— 20世纪60年代初）

- **1958年，罗森布拉特（F. Rosenblatt）**提出**感知机模型（perceptron）**。
- 1959年，**威德罗（B. Widrow）**等提出自适应线性元件（adaline）网络，通过训练后可用于抵消通信中的回波和噪声。1960年，他和**M. Hoff** 提出LMS（Least Mean Square 最小方差）算法的学习规则。



感知机的发明

- 弗兰克·罗森布拉特（Frank Rosenblatt），康奈尔大学主修社会心理学，并获得了心理学博士学位。
- 一台足有5吨重、面积大若一间屋子的IBM 704被“喂”进一系列的打孔卡。经过50次试验，计算机自己学会了识别卡片上的标记是在左侧还是右侧，这是对感知机（perceptron）的一次示范



罗森布拉特1958年夏季发表的《智能自动机的设计》一文中的感知机原理图。图片来源：康奈尔大学图书馆珍稀品和手稿收藏处（Rare and Manuscript Collections）

❑ 低潮时期：（20世纪60年代末— 20世纪70年代）

- 1969年，明斯基（M. Minsky）等在《Perceptron》中对感知器功能得出悲观结论。
- 1972年，T. Kohonen 和 J. Anderson 分别提出能完成记忆的新型神经网络。
- 1976年，S. Grossberg 在自组织神经网络方面的研究十分活跃。

❑ 第二次热潮时期： 20世纪80年代至今

- 1982年—1986年，霍普菲尔德（J. J. Hopfield）陆续提出离散的和连续的全互连神经网络模型，并成功求解旅行商问题（TSP）。
- 1986年，鲁姆尔哈特（Rumelhart）和麦克劳（McClelland）等在《Parallel Distributed Processing》中提出反向传播学习算法（BP算法）。
- 1987年6月，首届国际神经网络学术会议在美国圣地亚哥召开，成立了国际神经网络学会（INNS）。

马文·明斯基，“人工智能之父”和框架理论的创立者，是首批机械人手臂、世界上首位神经网络模拟器Snare、世界上最早能够模拟人类活动的机器人Robot C的创建者。同时，他还是世界上first人工智能实验室——MIT人工智能实验室的联合创始人，以及虚拟现实（VR）的最早倡导者。

明斯基和帕普特出版《感知机》一书，负面评价感知机，指出当前研究神经网络的模型是不可能的，计算时间如果不是无限的话，也是漫长的。

A Step toward the Understanding of Information Processes: Perceptrons. An Introduction to Computational Geometry. Marvin Minsky and Seymour Papert. M.I.T. Press, Cambridge, Mass., 1969. vi + 258 pp., illus. Cloth, \$12; paper, \$4.95.

ALLEN NEWELL

SCIENCE • 22 Aug 1969 • Vol 165, Issue 3895 • pp. 780-782 • DOI:10.1126/science.165.3895.780

Book Reviews

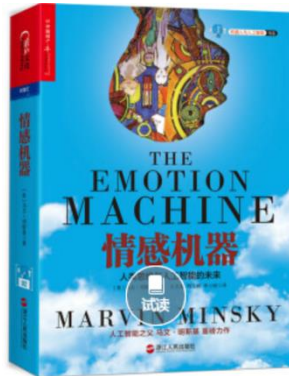
A Step toward the Understanding of Information Processes

Perceptrons. An Introduction to Computational Geometry. Marvin Minsky and Seymour Papert. M.I.T. Press, Cambridge, Mass., 1969. vi + 258 pp., illus. Cloth, \$12; paper, \$4.95.

This is a great book. To understand the perceptron, and what it can and cannot do, is to understand the limits of what can be done by a machine. For the book is about things that are not only possible but also probable. It is a book that should be read by anyone who is interested in the limits of what can be done by a machine. It is a book that should be read by anyone who is interested in the limits of what can be done by a machine. It is a book that should be read by anyone who is interested in the limits of what can be done by a machine.

more results are that there is no general algorithm for determining whether a given set of perceptrons can solve a given problem. This is a result that is as surprising as it is important. It is a result that is as surprising as it is important. It is a result that is as surprising as it is important.

more results are that there is no general algorithm for determining whether a given set of perceptrons can solve a given problem. This is a result that is as surprising as it is important. It is a result that is as surprising as it is important. It is a result that is as surprising as it is important.



神经网络的历史发展

- 在20世纪剩下的时间里，支持向量机和其它更简单的算法（如线性分类器）的流行程度逐步超过了神经网络。
- 1989年，Yann LeCun提出了一种用反向传导进行更新的卷积神经网络，称为LeNet。
- 1997年，Sepp Hochreiter和Jürgen Schmidhuber提出了长短期记忆网络。
- 1998年，以Yann LeCun为首的研究人员实现了七层的卷积神经网络LeNet-5，以识别手写数字。
- 21世纪初，借助GPU和分布式计算，计算机的计算能力大大提升。
- 2006年，Geoffrey Hinton用贪婪逐层预训练，有效训练了一个深度信念网络。以Geoffrey Hinton为代表的加拿大高等研究院附属机构的研究人员开始将人工神经网络/联结主义重新包装为了深度学习并进行推广。
- 2009-2012年，瑞士人工智能实验室IDSIA的Jürgen Schmidhuber带领研究小组发展了递归神经网络和深前馈神经网络。
- 2012年，Geoffrey Hinton组的研究人员在ImageNet 2012上夺冠，深度学习的热潮由此开始并一直持续到现在。

计算机学家、心理学家杰弗里·辛顿（Geoffrey Hinton）提出深度学习



Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton, *et al.*
Science **313**, 504 (2006);
DOI: 10.1126/science.1127647

The following resources related to this article are available online at www.sciencemag.org (this information is current as of October 6, 2008):

Updated information and services, including high-resolution figures, can be found in the online version of this article at:
<http://www.sciencemag.org/cgi/content/full/313/5786/504>

Supporting Online Material can be found at:
<http://www.sciencemag.org/cgi/content/full/313/5786/504/DC1>

A list of selected additional articles on the Science Web sites related to this article can be found at:

<http://www.sciencemag.org/cgi/content/full/313/5786/504#related-content>

This article **cites 8 articles**, 6 of which can be accessed for free:
<http://www.sciencemag.org/cgi/content/full/313/5786/504#otherarticles>

This article has been **cited by** 15 article(s) on the ISI Web of Science.

This article has been **cited by** 4 articles hosted by HighWire Press; see:
<http://www.sciencemag.org/cgi/content/full/313/5786/504#otherarticles>

Information about obtaining **reprints** of this article or about obtaining **permission to reproduce this article** in whole or in part can be found at:
<http://www.sciencemag.org/about/permissions.shtml>



Geoffrey Hinton

2018年图灵奖

2024年诺贝尔物理学奖

深度学习的开篇：2006年. G.E. Hinton 和R. R. Salakhutdinov的 “Reducing the dimensionality of data with neural networks”

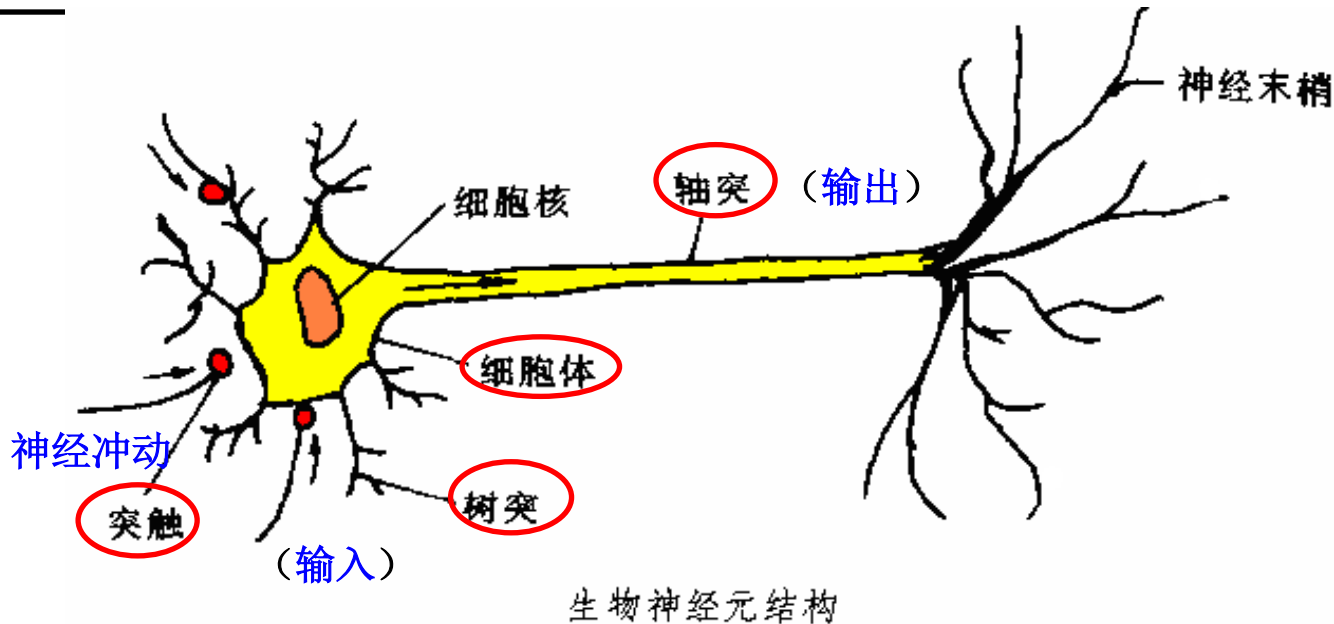
- 人脑由一千多亿（ 10^{11} 亿— 10^{14} 亿）个神经细胞（神经元）交织在一起的网状结构组成，其中大脑皮层约140亿个神经元，小脑皮层约1000亿个神经元。



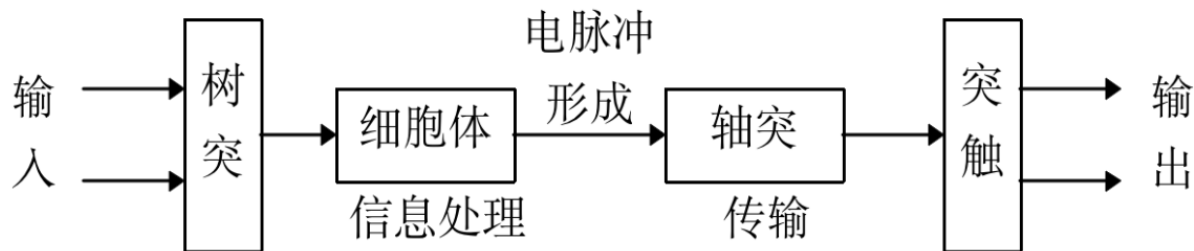
- 神经元约有1000种类型，每个神经元大约与 10^3 — 10^4 个其他神经元相连接，形成极为错综复杂而又灵活多变的神经网络。
- 人的智能行为就是由如此高度复杂的组织产生的。浩瀚的宇宙中，也许只有包含数千亿颗星球的银河系的复杂性能够与大脑相比。



生物神经元的结构



- 神经元的信息传递和处理是一种电化学反应，**树突**由于电化学反应接受外界的刺激(**输入**)；通过胞体内的活动体现为**轴突**电位，当轴突电位达到一定的值则形成神经脉冲或动作电位；再通过轴突末梢传递给其他神经元(**输出**)。
- 从控制论的观点来看，这一过程可以看作一个多输入单输出非线性系统的动态过程。



- 神经元的工作状态：
 - **兴奋状态**：细胞膜电位 $>$ 动作电位的阈值 $\rightarrow$ 神经冲动
 - **抑制状态**：细胞膜电位 $<$ 动作电位的阈值
- 学习与遗忘：由于神经元结构的可塑性，突触的传递作用可增强和减弱。
- 脑神经信息活动的特征：
 - 巨量并行性。
 - 信息处理和存储单元结合在一起。
 - 自组织自学习功能。

人工神经元模型可以看成是由三种基本元素组成：

(1) 一组连接

连接强度由各连接上的权值表示，权值可以取正值也可以取负值，权值为正表示激活，权值为负表示抑制；

(2) 一个加法器

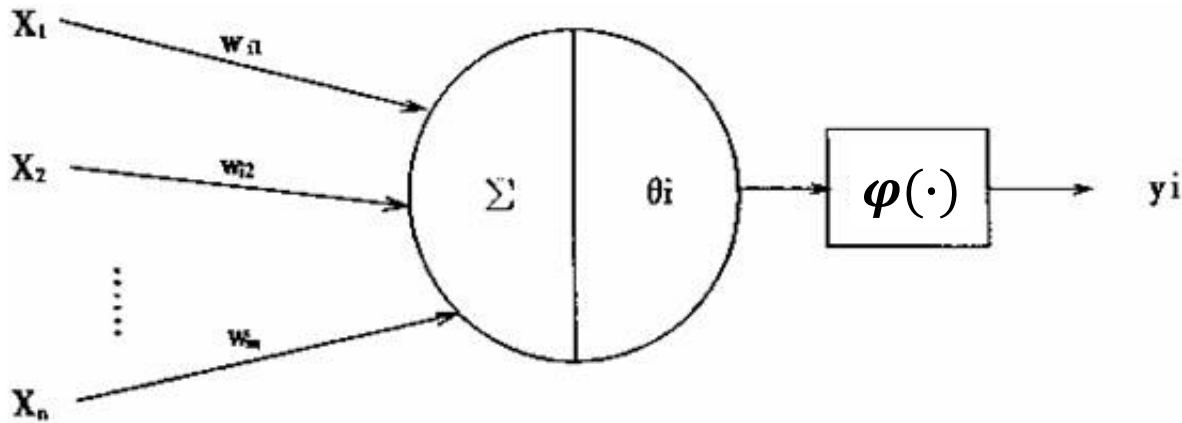
用于求输入信号对神经元的相应突触加权之和；

(3) 一个激活函数

用限制神经元输出振幅，激活函数也称为压制函数。因为他将输入信号压制（限制）到允许范围之内的一定值。



输入信号 连接权重 求和 阈值 激活函数 输出



图中 $x_1, x_2 \cdots x_n$ 为神经元的输入，即是来自前级n个神经元的轴突的信息，是i神经元的阈值； $w_{i1}, w_{i2} \cdots w_{in}$ 分别是i神经元对 $x_1, x_2 \cdots x_n$ 的权重系数，也即突触的传递效率； y_i 是i神经元的输出； $\varphi(\cdot)$ 是激活函数，它决定i神经元受到输入 $x_1, x_2 \cdots x_n$ 的共同刺激达到阈值时以何种方式输出。

生物神经元具有信息整合能力，为实现类似功能，人工神经元将所有输入 $x_1, x_2, \dots, x_m$ 进行加权求和实现信息整合，即有：

$$I = \sum_{i=1}^m w_i x_i$$

人工神经元的输出定义为：

$$y = \varphi \left(\sum_{i=1}^m w_i x_i - \theta_i \right)$$

其中 θ 为给定阈值， φ 表示激活函数



- 神经网络概述
- 神经元结构
- 多层神经网络
- 激活函数
- 梯度下降
- 反向传播算法
- 神经网络训练方法



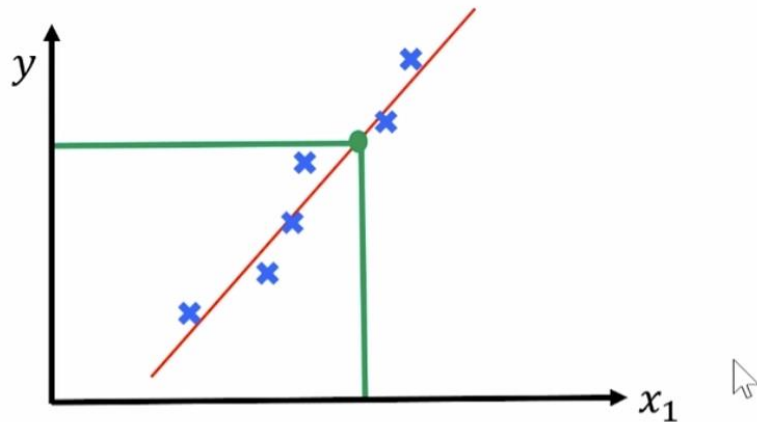
回顾：单变量线性回归模型

- 线性回归可以找到一些点的集合背后的规律：一个点集可以用一条直线来拟合，这条拟合出来的直线的参数特征，就是线性回归找到的点集背后的规律。

单变量线性模型

$$H_w(x) = w_0 + wx$$

x : Feature; $H(x)$: hypothesis



回顾：多变量线性回归模型

假设影响售价 y 的特征不仅仅只有房屋面积 x_1 ，还有楼层 x_2 ，房屋朝向

x_3 , x_4 , ... , x_n

单变量线性模型

$$H_w(x) = w_0 + wx$$



多变量线性模型

$$H_w(x) = w_0 + w_1x_1 + w_2x_2 \quad \text{2个特征}$$



$$H_w(x) = \sum_{i=0}^n w_i x_i = \hat{\mathbf{w}}^T \mathbf{x}, \quad \text{n个特征}$$

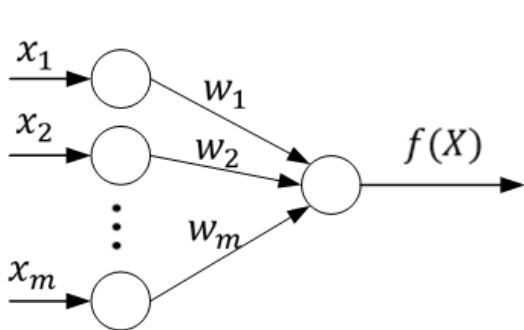
$$\hat{\mathbf{w}} = [w_0; w_1; \dots w_n],$$

$$\mathbf{x} = [x_0; x_1; \dots x_n], \quad x_0 = 1$$

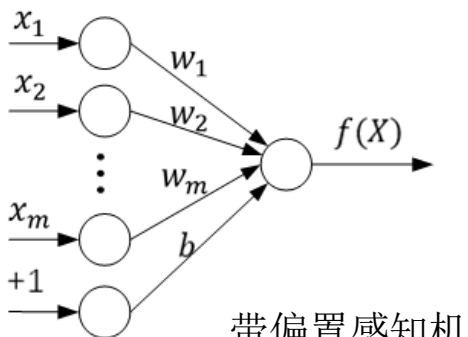
感知机模型

感知机模型是一种只有一层神经元参与数据处理的人工神经网络模型，感知机模型通过输入层接受输入信息 $X = (x_1, x_2, \dots, x_m)^T$ ，感知机模型的输出为： $f(X) = \text{sgn}(\sum_{i=1}^m w_i x_i + b)$ ，即

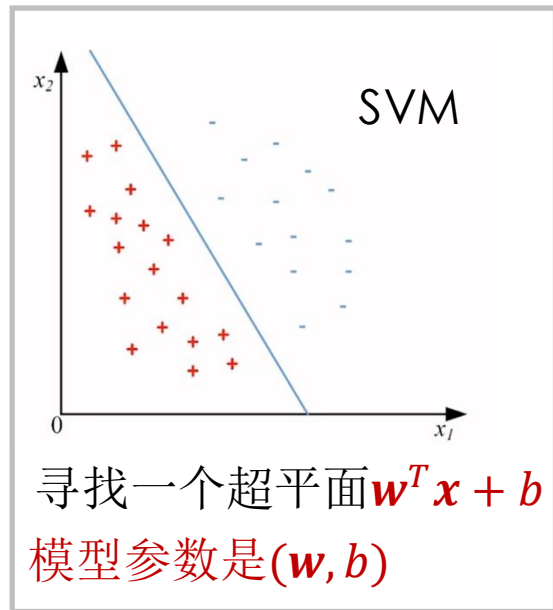
$$f(X) = \text{sgn}(w^T x + b), \quad \text{sgn}(x) = \begin{cases} +1 & x \geq 0 \\ -1 & x \leq 0 \end{cases}$$



感知机模型



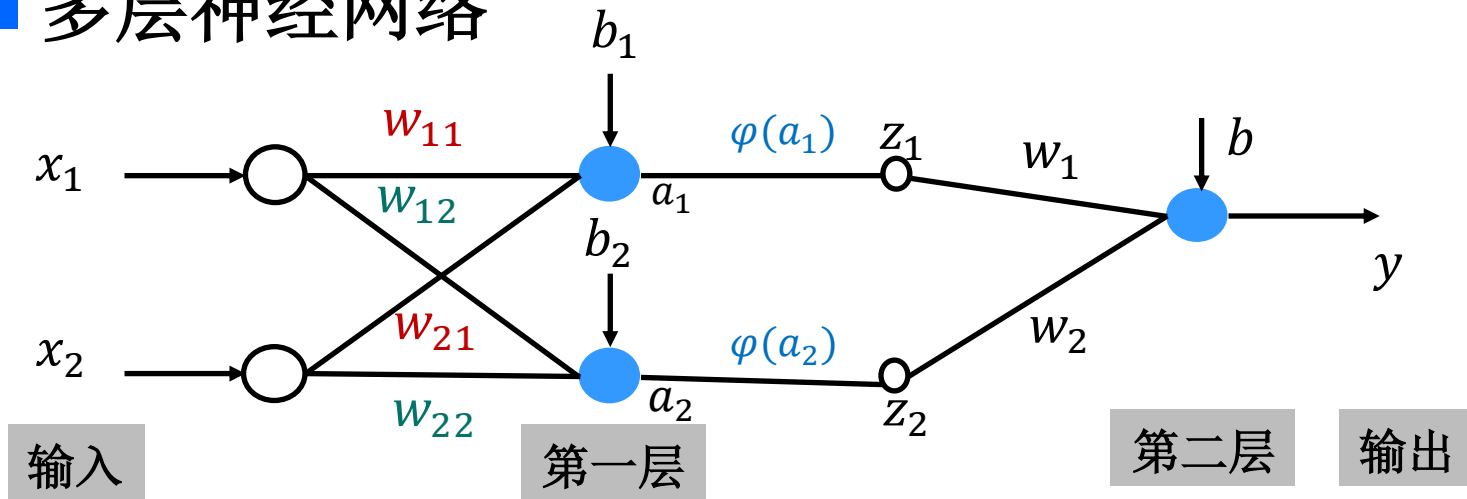
带偏置感知机





两层神经网络示例

■ 多层神经网络



$$\begin{cases} a_1 = w_{11}x_1 + w_{21}x_2 + b_1 \\ a_2 = w_{12}x_1 + w_{22}x_2 + b_2 \\ z_1 = \varphi(a_1) \\ z_2 = \varphi(a_2) \end{cases}$$

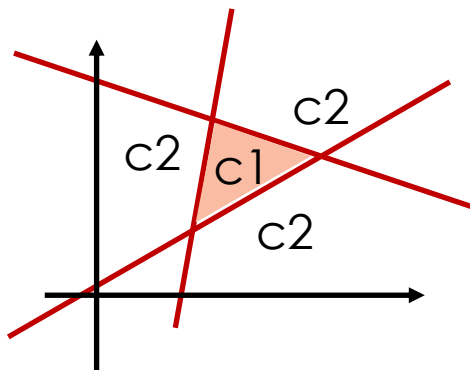
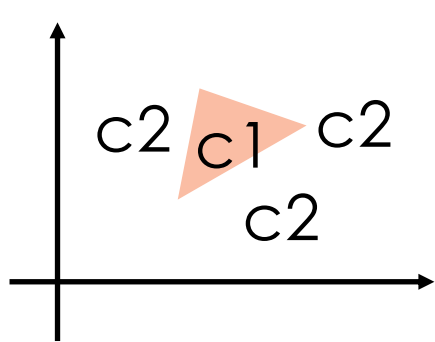
$$y = w_1z_1 + w_2z_2 + b$$

$\varphi(\cdot)$ 是一个非线性函数





两层神经网络

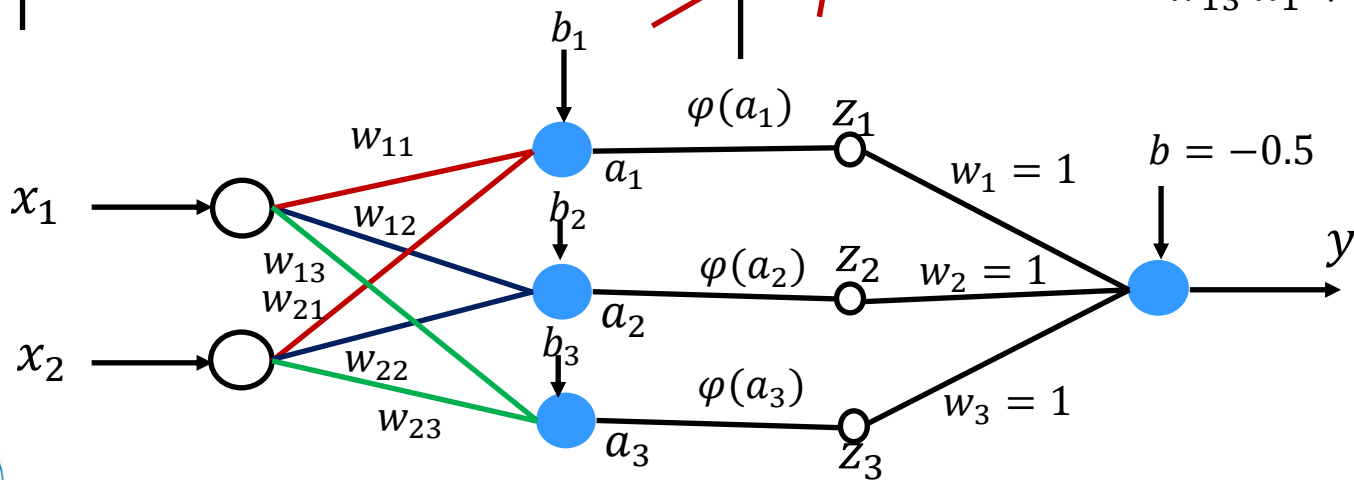


三条线:

$$w_{11} x_1 + w_{21} x_2 + b_1 = 0$$

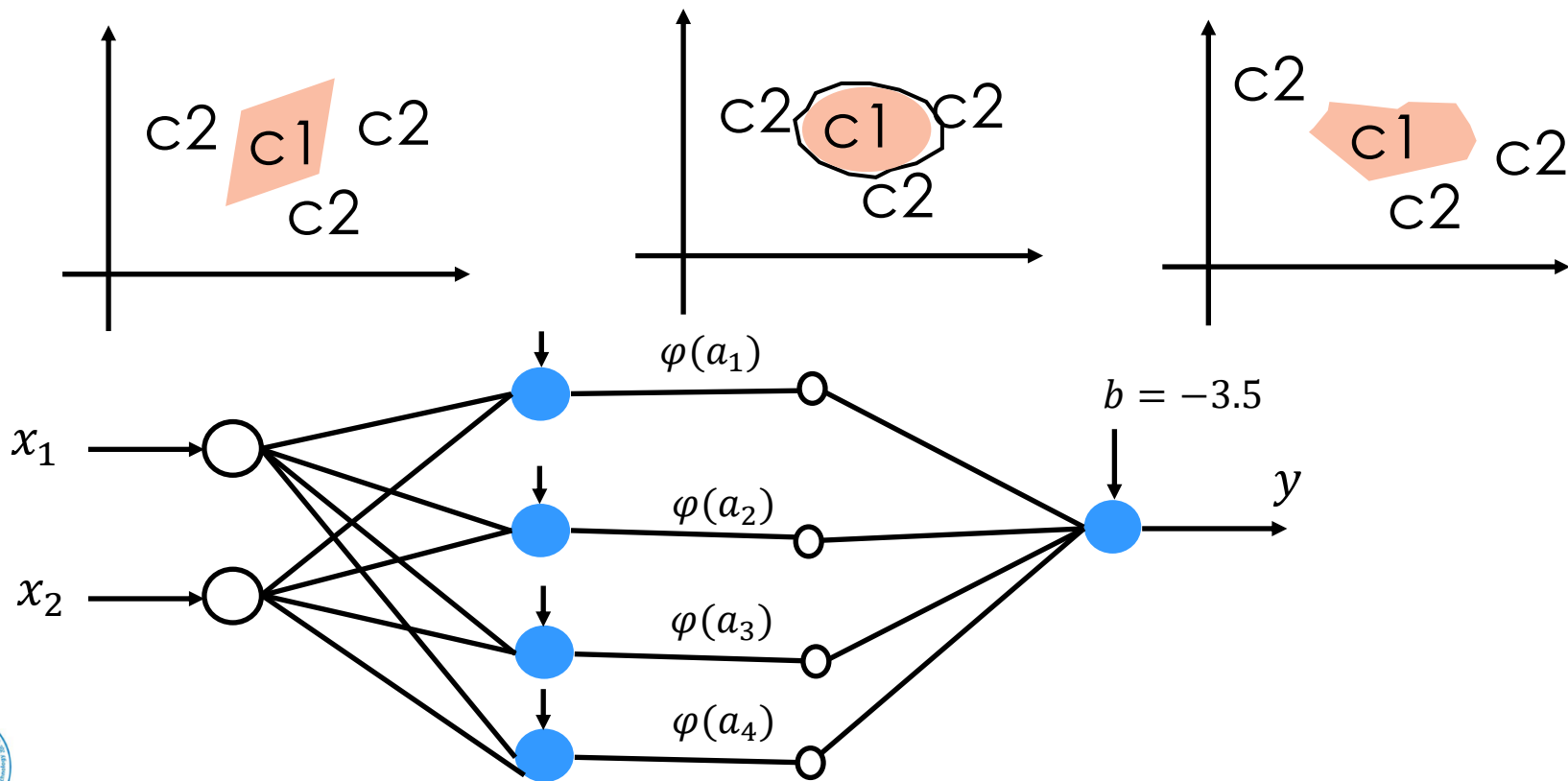
$$w_{12} x_1 + w_{22} x_2 + b_2 = 0$$

$$w_{13} x_1 + w_{23} x_2 + b_3 = 0$$

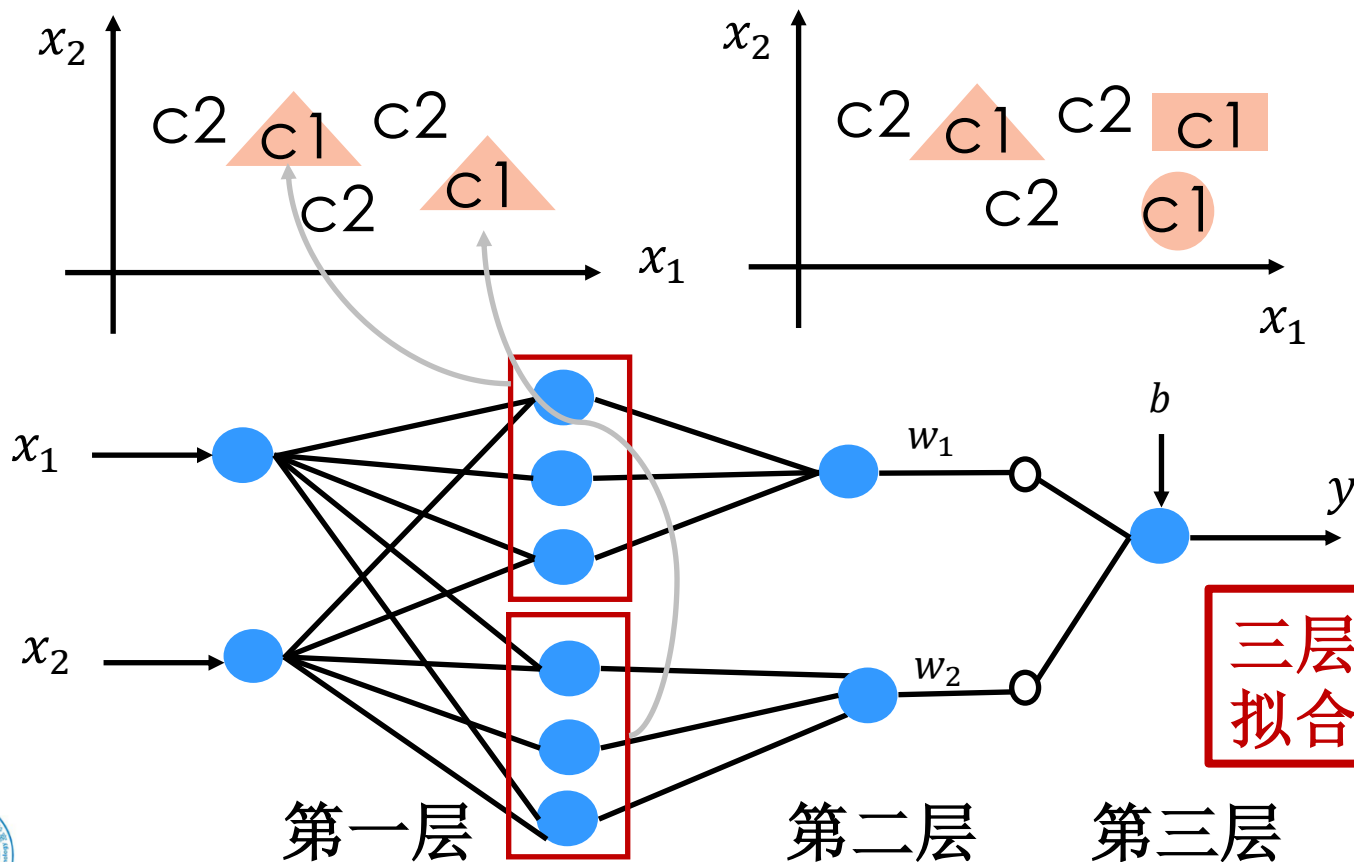




拟合任意形状



拟合任意决策面



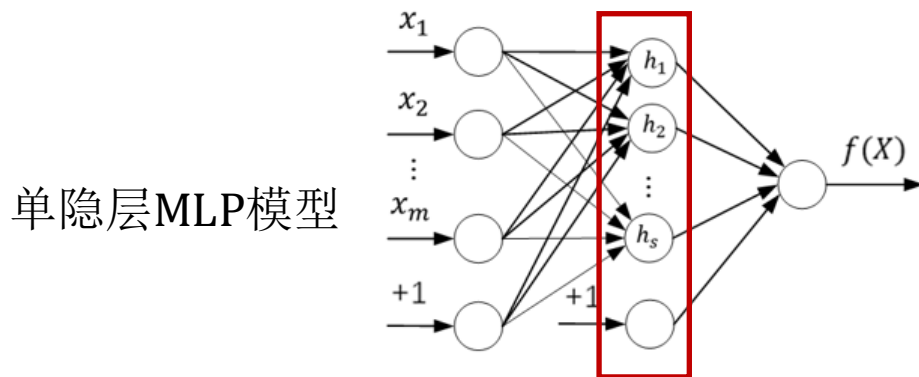
上面3个

然后4个

接着很多个

三层神经网络可
拟合任何决策面

- 通常将此类没有环路或回路的人工神经网络称为前馈网络模型。
- 感知机是一种最简单的前馈网络模型，在感知机模型的基础之上添加隐含层，通常将此类模型称为多层感知机模型（MLP）。



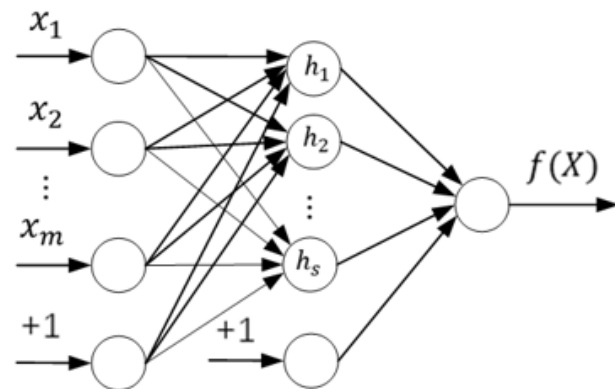
MLP模型中隐含层的层数可为一层也可为多层。隐含层可将数据通过非线性映射表示在另一个空间当中，并将模型输出限制在区间(0,1)当中。

MLP模型中信息处理神经元的激活函数通常为Sigmoid函数。

假设右图所示神经网络模型的隐含层包含 s 个神经元，输入向量为： $\mathbf{X} = (x_1, x_2, \dots, x_m)^T$ ，

模型第 l 层第 i 个结点与下一层的第 j 个结点之间的连接权重为 $w_{ij}^{(l)}$ ，第 l 层第 i 个结点的偏置为 $b_i^{(l)}$ ，则隐含层第 v 个结点的输出为：

$$h_v = \varphi_h \left(\sum_{u=1}^m w_{uv}^{(1)} x_u + b_v^{(2)} \right), \text{ 其中 } \varphi_h \text{ 表示sigmoid函数}$$

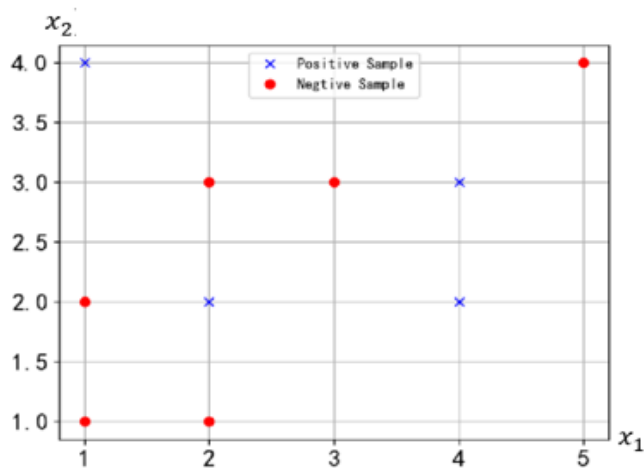


令隐含层的输出向量为 $\mathbf{h}' = (h_1, h_2, \dots, h_s)^T$ ，则可将上述计算公式表示为向量形式。将MLP模型模型输入层与隐含层之间的权重矩阵表示为 $\mathbf{W}^1 = (w_1^{(1)}, w_2^{(1)}, \dots, w_s^{(1)})^T$ 。由此可将隐含层的输出向量 $\mathbf{h}'$ 表示为如下形式：

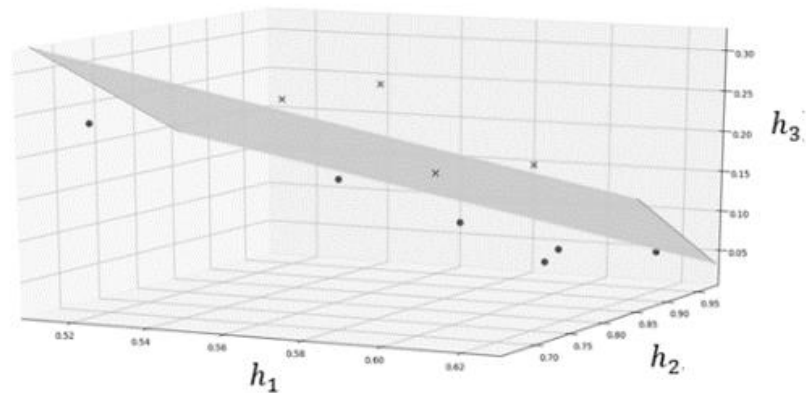
$$\mathbf{h}' = \left(\varphi_h(\mathbf{w}_1^{(1)T} \mathbf{X}), \varphi_h(\mathbf{w}_2^{(1)T} \mathbf{X}), \dots, \varphi_h(\mathbf{w}_s^{(1)T} \mathbf{X}) \right)^T$$

相当于将 m 维样本通过非线性映射表示在 s 维空间中

以输入向量为 $X = (x_1, x_2)^T$ 且隐含层输出向量为 $\mathbf{h}' = (h_1, h_2, h_3)^T$ 的模型为例，使用训练数据集 D 构造模型， D 中原始数据分布情况如左图所示，使用该数据集训练好的MLP模型可通过隐含层将原始数据分布映射为如右图所示的分布，可见**原线性不可分的数据映射后线性可分**。



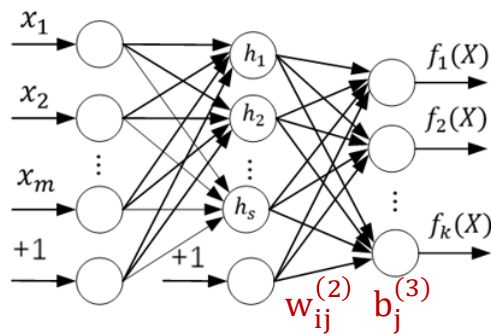
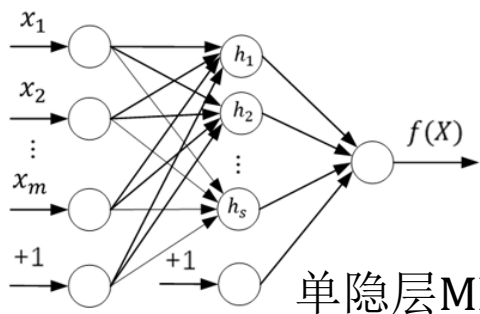
原始数据分布



映射后数据分布

多层感知机模型

由于MLP模型输出层只有一个神经元，故只适用于二分类任务。对于多分类任务，可考虑增加输出层神经元个数，使得模型具备处理多分类任务的能力，由此得到一种如下图所示的反向传播神经网络模型或BP(Back Propagation)神经网络模型。



现考虑输出层处理过程。令 $\mathbf{h} = (1, h_1, h_2, \dots, h_s)^T$ ，将 $\mathbf{h}$ 作为输入向量交由输出层进行处理，可得输出层第 j 个神经元的输出为：

$$f_j(X) = \varphi \left(\sum_{i=1}^s w_{ij}^{(2)} h_i + b_j^{(3)} \right)$$

表示为向量形式 $f_j(X) = \sigma(\mathbf{w}_i^{(2)T} \mathbf{h})$ ，其中 $\mathbf{w}_i^{(2)}$ 为与输出层中第 i 个结点相关的连接权重所组成的向量。

- 神经网络是**并行化**系统
- **非线性**: 线性系统是满足叠加原理和齐次性的系统, 例如, $y=kx$ 这样的函数关系是线性, 而神经网络是典型的非线性系统, 不满足这两个特性, 也正因为它是非线性系统, 所以才能解决非线性分类等问题。
- **非常定性**: 定常就是一个系统的结构参数是固定的常数, 而神经网络是可以通过训练修改权值的, 当然具有非定常性。
- **非凸性**: 非凸性是指系统的能量函数有多个极值, 即系统有多个稳定的平衡态。

如何选择合适的神经网络, 构造合理的学习算法, 由实验获得, 理论并不完备



- 神经网络概述
- 神经元结构
- 多层神经网络
- 激活函数
- 梯度下降
- 反向传播算法
- 神经网络训练方法

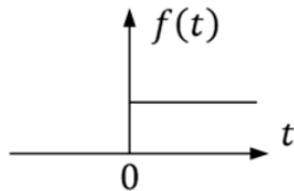


■ 非线性函数 $\varphi(\cdot)$

激活函数 φ 的作用是对神经元的输出加以限制使得其有界，通常将输出范围限制在 $[0,1]$ 或 $[-1,1]$ 。常用的激活函数主要有阶跃函数、Sigmoid函数等。

分段函数就是一种常用阶跃函数：

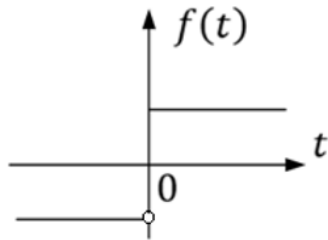
$$\varphi(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$



阶跃函数的输出只能取0或1，分别表示该神经元处于抑制或兴奋状态

若将神经元处于抑制状态表示为-1，即有：

$$\varphi(t) = \text{sgn}(t) = \begin{cases} 1, & t \geq 0 \\ -1, & t < 0 \end{cases}$$



$$\text{求导} \varphi'(t) = 0$$

Sigmoid函数是一种最常用的激活函数，可将人工神经元的输出限制在区间(0,1)内

$$\varphi(t) = \text{Sigmoid}(t) = \frac{1}{1 + e^{-t}} \quad \varphi'(t) = \varphi(t)[1 - \varphi(t)]$$

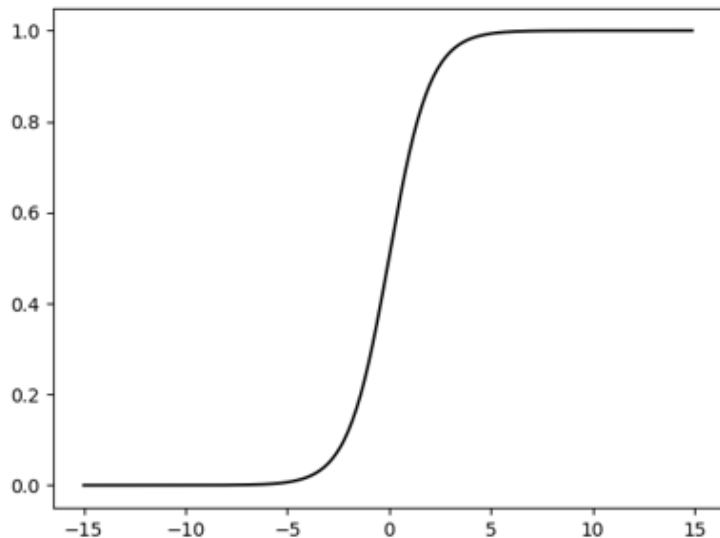
可对Sigmoid函数进行适当改造以调整输出范围，获得新的激活函数。例如，令：

$$\varphi(t) = 2\text{Sigmoid}(t) - 1$$

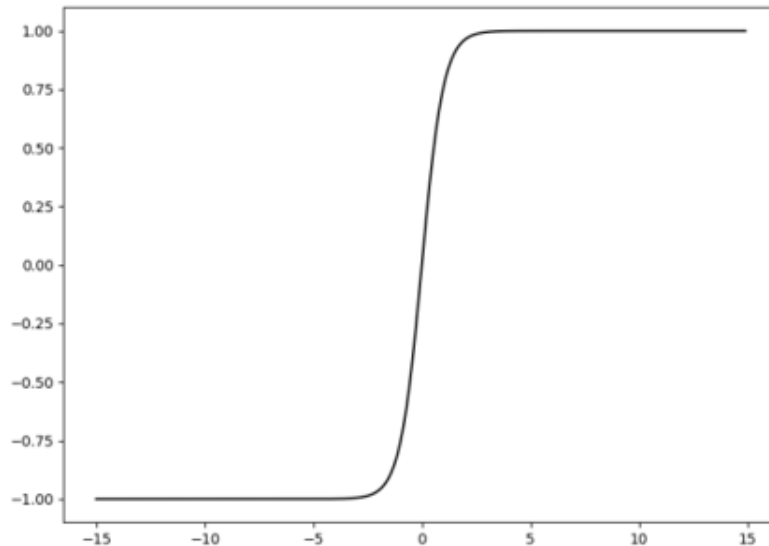
则可得到一个输出范围是(-1,1)的激活函数，通常称该激活函数为 **tanh 函数**，tanh函数的具体形式如下：

$$\varphi(t) = \tanh(t) = \frac{e^t - e^{-t}}{e^t + e^{-t}} \quad \varphi'(t) = 1 - [\varphi(t)]^2$$

$\tanh$ 函数将Sigmoid函数图像在竖直方向上拉伸了两倍并向下平移了一个单位，有时会更便于进行模型参数求解。



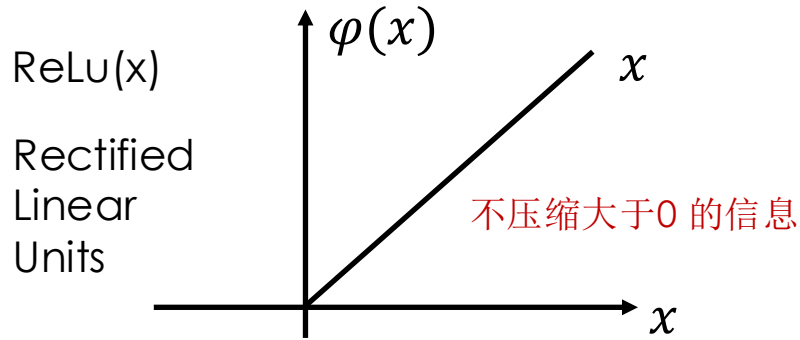
Sigmoid激活函数



$\tanh$ 激活函数

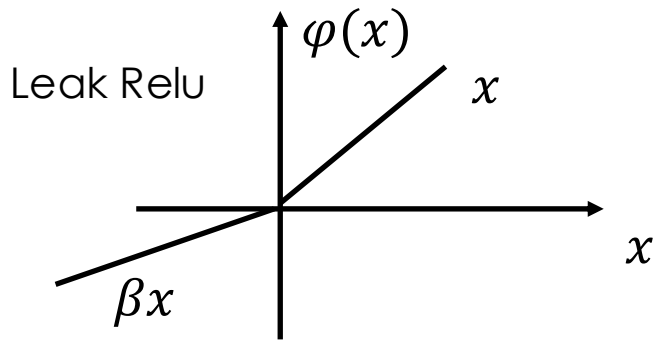


深度学习中常用的激活函数



$$\varphi(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} = \max(0, x)$$

求导 $\varphi'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$



$$\varphi(x) = \begin{cases} x, & x > 0 \\ \beta x, & x \leq 0 \end{cases}$$

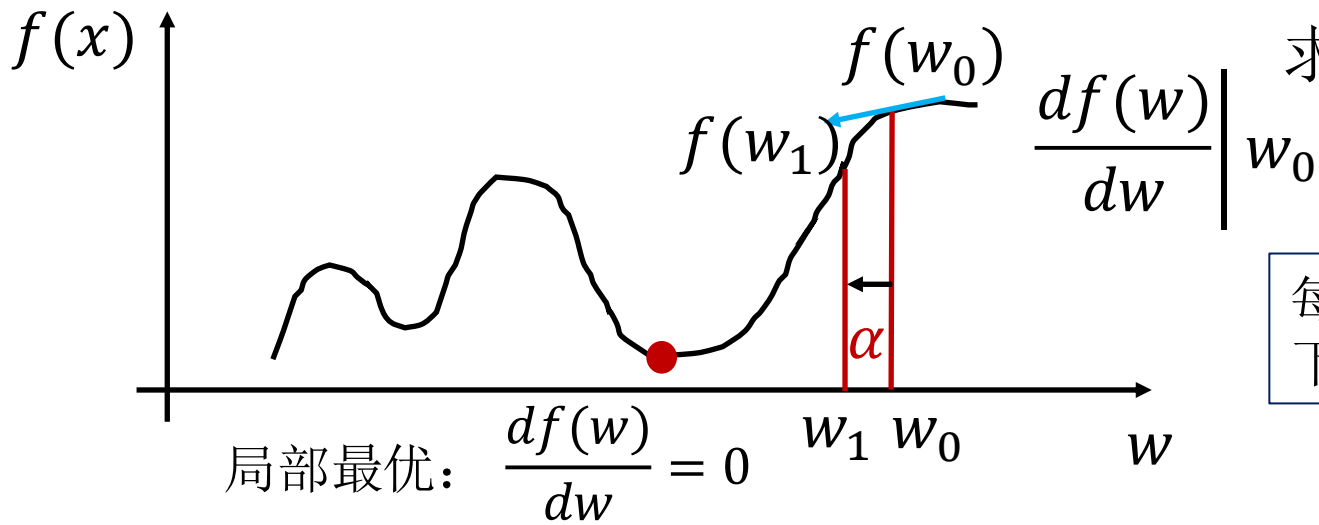
求导 $\varphi'(x) = \begin{cases} 1, & x > 0 \\ \beta, & x \leq 0 \end{cases}$



- 神经网络概述
- 神经元结构
- 多层神经网络
- 激活函数
- 梯度下降
- 反向传播算法
- 神经网络训练方法



梯度下降基本原理



每个点上，找一个下降最快的方向

算法过程：

(1) 找一个 w_0

(2) 设 $k=0$ ，若 $\frac{df(w)}{dw} = 0$ ，退出

否则 $w_{k+1} = w_k - \alpha \left. \frac{df(w)}{dw} \right|_{w_k}$

α : 学习率

更新公式，可找到局部最优



证明:

$$f(w + \Delta w) = f(w) + \frac{df(w)}{dw} \Delta w + o(\Delta w)$$

$$f(w_{k+1}) = f(w_k) + \frac{df(w)}{dw} \Big|_{w_k} \left(-\alpha \frac{df(w)}{dw} \Big|_{w_k} \right) + o(\Delta w)$$

$$= f(w_k) - \alpha \left[\frac{df(w)}{dw} \Big|_{w_k} \right]^2 + o(\Delta w) < f(w_k)$$

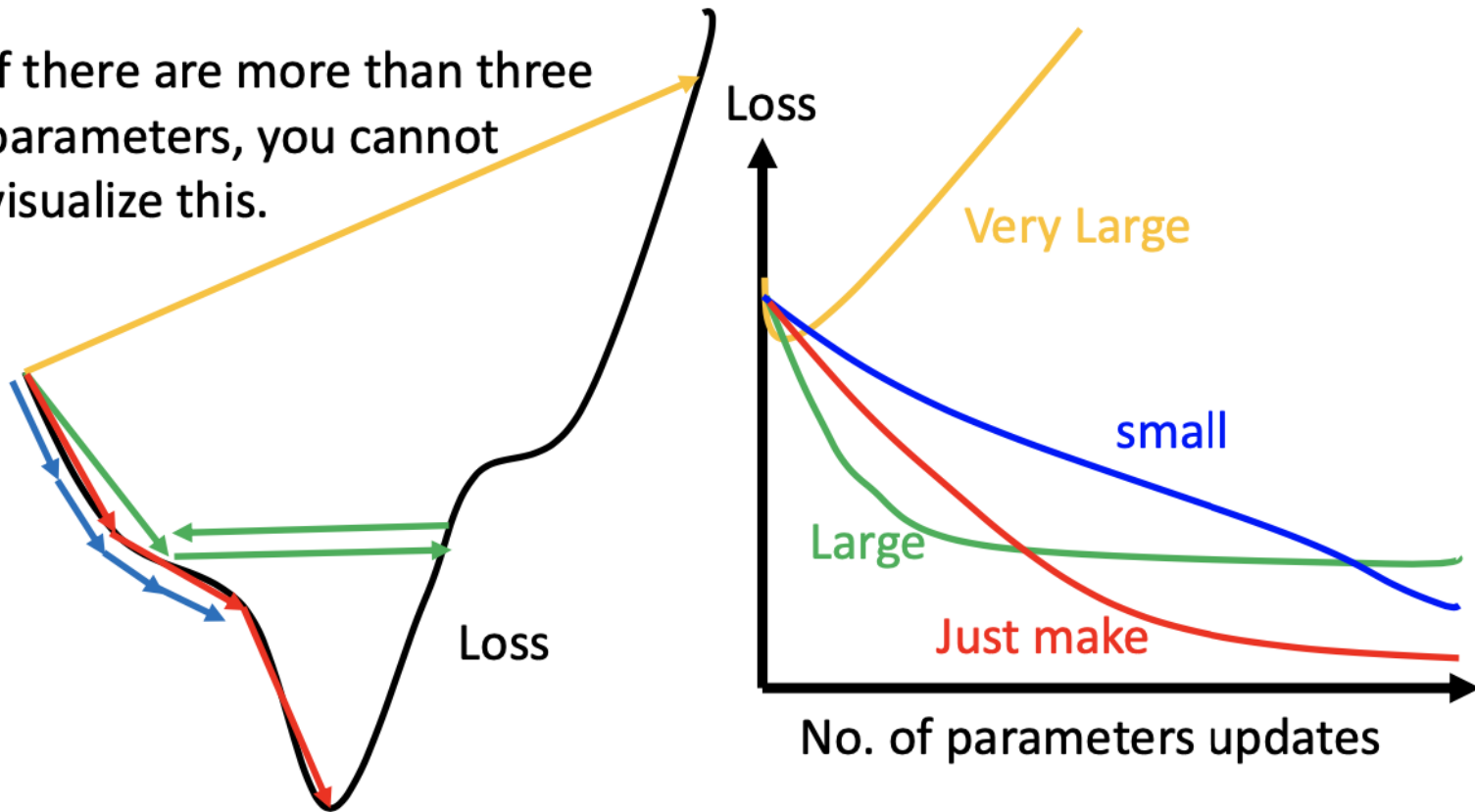
w 足够小, 则这部分小于0

因此, 更新之后的 $f(w_{k+1})$ 一定小于 $f(w_k)$



关于 α

If there are more than three parameters, you cannot visualize this.



自适应学习率：先快后慢

优化问题

$$\theta^* = \arg \min_{\theta} L(\theta) \quad L: \text{loss function} \quad \theta: \text{parameters}$$

Suppose that θ has two variables $\{\theta_1, \theta_2\}$

Randomly start at $\theta^0 = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix}$

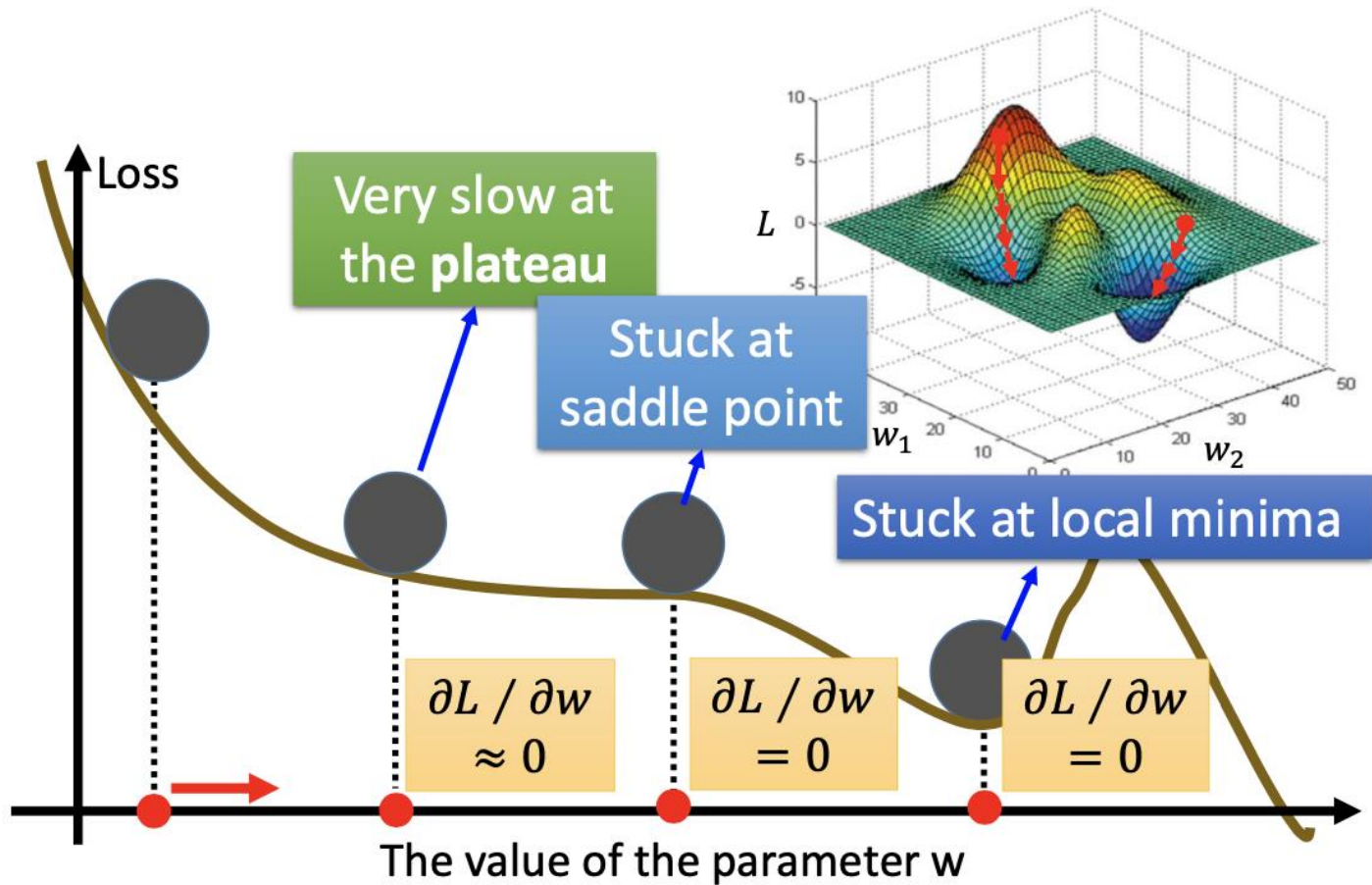
$$\nabla L(\theta) = \begin{bmatrix} \partial L(\theta_1)/\partial \theta_1 \\ \partial L(\theta_2)/\partial \theta_2 \end{bmatrix}$$

第一次更新 $\begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix} - \eta \begin{bmatrix} \partial L(\theta_1^0)/\partial \theta_1 \\ \partial L(\theta_2^0)/\partial \theta_2 \end{bmatrix} \Rightarrow \theta^1 = \theta^0 - \eta \nabla L(\theta^0)$

第二次更新 $\begin{bmatrix} \theta_1^2 \\ \theta_2^2 \end{bmatrix} = \begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} - \eta \begin{bmatrix} \partial L(\theta_1^1)/\partial \theta_1 \\ \partial L(\theta_2^1)/\partial \theta_2 \end{bmatrix} \Rightarrow \theta^2 = \theta^1 - \eta \nabla L(\theta^1)$



梯度下降示例





- 神经网络概述
- 神经元结构
- 多层神经网络
- 激活函数
- 梯度下降
- 反向传播算法
- 神经网络训练方法

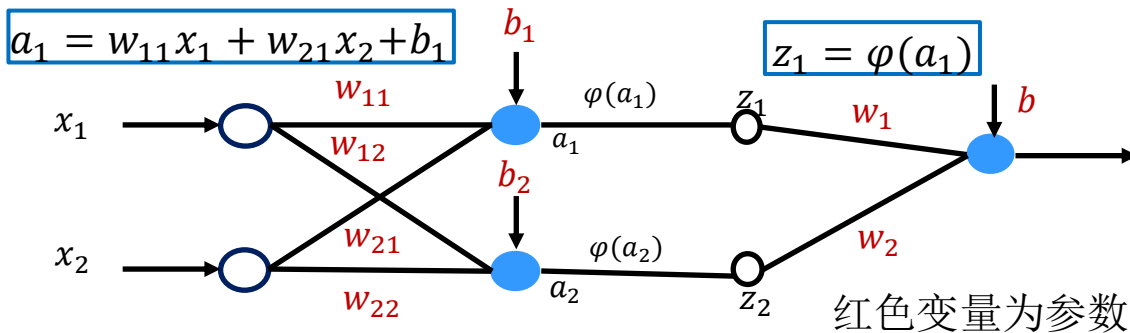




反向传播示例

数据集: $\{(x_i, x_j)\}_{i=1 \sim N}$, 对样本 (X, Y) , 求 $E = \frac{1}{2}(y - Y)^2$

$$y = w_1 z_1 + w_2 z_2 + b$$



$$\frac{dE}{dy} = (y - Y)$$

$$\frac{\partial E}{\partial b} = \frac{dE}{dy} \cdot \frac{\partial y}{\partial b} = (y - Y)$$

$$\frac{\partial E}{\partial w_1} = \frac{dE}{dy} \cdot \frac{\partial y}{\partial w_1} = (y - Y)z_1$$

$$\frac{\partial E}{\partial w_{11}} = \frac{\partial E}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{11}} = (y - Y)w_1 \phi'(a_1)x_1$$

$$\frac{\partial E}{\partial a_1} = \frac{dE}{dy} \cdot \frac{\partial y}{\partial z_1} \cdot \frac{dz_1}{da_1} = (y - Y)w_1 \phi'(a_1)$$

$$\frac{\partial E}{\partial a_2} = \frac{dE}{dy} \cdot \frac{\partial y}{\partial z_2} \cdot \frac{dz_2}{da_2} = (y - Y)w_2 \phi'(a_2)$$

$$\frac{\partial E}{\partial b_1} = \frac{\partial E}{\partial a_1} \cdot \frac{\partial a_1}{\partial b_1} = (y - Y)w_1 \phi'(a_1) \dots$$

(1) 随机设置所有参数值

(2) 对所有 w 求 $\frac{\partial E}{\partial w}$, 对所有 b 求 $\frac{\partial E}{\partial b}$

(3) $w^{new} = w^{old} - \alpha \left. \frac{\partial E}{\partial w} \right|_{w^{old}}$ $b^{new} = b^{old} - \alpha \left. \frac{\partial E}{\partial b} \right|_{b^{old}}$

(4) 当所有 $\frac{\partial E}{\partial w} = 0$, 所有 $\frac{\partial E}{\partial b} = 0$, 退出

(1) 随机初始化(w, b)

前向传播

(2) 训练样本 (X, Y) 代入神经网络可以求得所有 (z, a, y)

(3) 链式法则求偏导:

$$\min E = \frac{1}{2} \|y - Y\|^2 = \frac{1}{2} \sum_{i=1}^M (y_i - Y)^2$$

反向求偏导

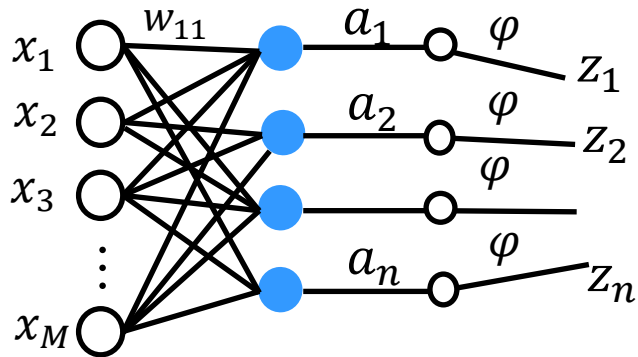
求 $\frac{\partial E}{\partial w}$ $\frac{\partial E}{\partial b}$

参数更新

(4) 更新: $w^{new} = w^{old} - \alpha \frac{\partial E}{\partial w} \Big|_{w^{old}}$ $b^{new} = b^{old} - \alpha \frac{\partial E}{\partial b} \Big|_{b^{old}}$



通用形式



神经网络共 l 层

$Z^{(k)}$ 、 $a^{(k)}$ 、 $b^{(k)}$ 是第 k 层的向量，与第 k 层神经元个数一致

$Z_i^{(k)}$ 、 $a_i^{(k)}$ 、 $b_i^{(k)}$ 表示 $Z^{(k)}$ 、 $a^{(k)}$ 、 $b^{(k)}$ 是第 i 个分量
 y_i 表示 y 的第 i 个分量

$$X = Z^{(0)} \quad W^{(1)}Z^{(0)} + b^{(1)} = a^{(1)} \xrightarrow{\varphi} Z^{(1)} = \varphi(a^{(1)})$$

$$\longrightarrow a^{(2)} = W^{(2)}Z^{(1)} + b^{(2)} \xrightarrow{\varphi} Z^{(2)} = \varphi(a^{(2)})$$

$$\longrightarrow a^{(3)} = W^{(3)}Z^{(2)} + b^{(3)} \quad \dots \quad a^{(m)} = W^{(m)}Z^{(m-1)} + b^{(m)}$$

$$a^{(l)} = W^{(l)}Z^{(l-1)} + b^{(l)}$$

$$y = Z^{(l)} = \varphi(a^{(l)})$$



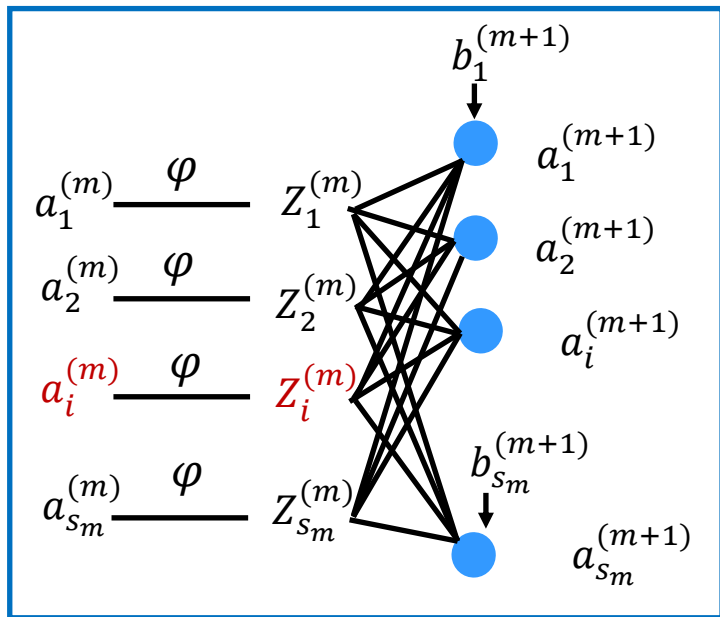


通用形式

$$a^{(m)} = W^{(m)} Z^{(m-1)} + b^{(m)}$$

$$Z^{(m)} = \varphi(a^{(m)})$$

$$a^{(m+1)} = W^{(m+1)} Z^{(m)} + b^{(m+1)}$$



$$\text{设 } \delta_i^{(m)} = \frac{\partial E}{\partial a_i^{(m)}}$$

$$(1) \quad \delta_i^{(l)} = \frac{\partial E}{\partial a_i^{(l)}} = \frac{\partial E}{\partial y_i} \cdot \frac{\partial y_i}{\partial a_i^{(l)}} = (y_i - Y_i) \varphi'(a_i^{(l)}) \quad \text{输出层}$$

$$(2) \quad 1 \leq m \leq l-1$$

$$\delta_i^{(m)} = \frac{\partial E}{\partial a_i^{(m)}} = \frac{\partial E}{\partial z_i^{(m+1)}} \cdot \frac{\partial z_i^{(m+1)}}{\partial a_i^{(m)}} = \varphi'(a_i^{(m)}) (\sum_{j=1}^{s_{m+1}} \delta_j^{(m+1)} w_{ji})$$

隐藏层

$$\left\{ \begin{array}{l} \frac{\partial E}{\partial w_{ij}^{(m)}} = \delta_j^{(m)} Z_j^{(m-1)} \\ \frac{\partial E}{\partial b_i^{(m)}} = \delta_i^{(m)} \end{array} \right.$$



- 神经网络概述
- 神经元结构
- 多层神经网络
- 梯度下降
- 反向传播算法
- 神经网络训练方法

- 一般情况下，在训练集上的目标函数的平均值（loss）随着训练的深入而不断减小，如果这个指标有增大情况，停下来。原因：
 - 模型不够复杂，不能在训练集上完全拟合
 - 已经训练很好了
- 分出验证集（Validation Set）：训练一段时间后，在验证集上进行测试，使验证集上识别率最大的模型参数，作为最后结果
- 调整学习率（Learning Rate），如果一开始就loss增加，则说明学习率过高，如果每次loss变化很小，则学习率过低。

- Batch大小设置
- 动量优化
- 训练数据的初始化
- (w, b) 的选择
- Batch Normalization
- 目标函数选择

- 批量梯度下降法BGD(**Batch** Gradient Descent): 针对的是整个数据集, 通过对所有的样本的计算来求解梯度的方向。

优点: 全局最优解; 易于并行实现;

缺点: 当样本数据很多时, 计算量开销大, 计算速度慢

- 小批量梯度下降法**MBGD** (mini-batch Gradient Descent) **把数据分为若干个批, 求出这些样本的梯度平均值**, 根据这个平均值改变参数, 一个批中的一组数据共同决定了本次梯度的方向, 下降起来就不容易跑偏, 减少了随机性

优点: 减少了计算的开销量, 降低了随机性

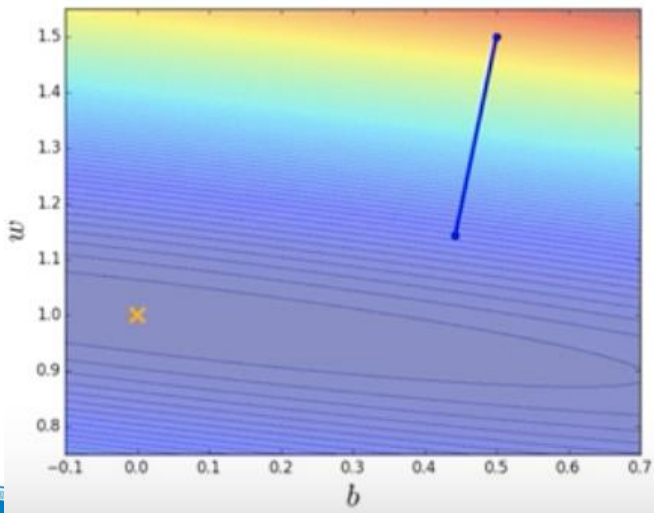
- 随机梯度下降法SGD (stochastic gradient descent): 每个数据都计算一下损失函数, 然后求梯度更新参数。

优点: 计算速度快 & 缺点: 收敛性能不好

假设数据集中20个样本

Batch_size=20 (full batch)

20个样本计算后，更新参数

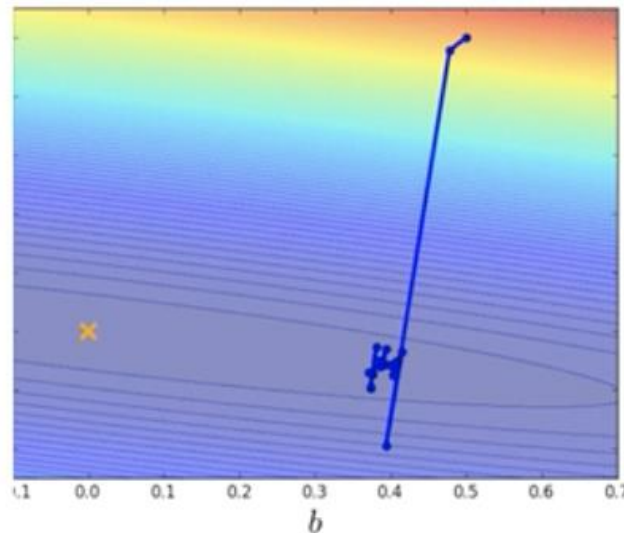


所有样本

假设数据集中20个样本

Batch_size=1 (full batch)

每个样本计算后，更新参数



1个样本

所有样本

Batch大小设置

SGD可视为MBGD的一个特例，及batch_size=1的情况。在深度学习及机器学习中，基本上都是使用的MBGD算法。

$$\theta^* = \arg \min_{\theta} L$$

➤ (Randomly) Pick initial values θ^0

➤ Compute gradient $g^0 = \nabla L^1(\theta^0)$ L^1

$$\text{update } \theta^1 \leftarrow \theta^0 - \eta g^0$$

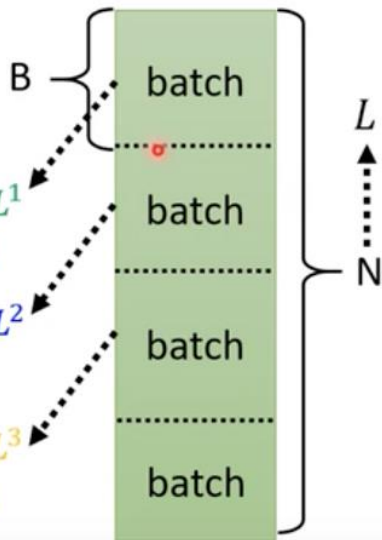
➤ Compute gradient $g^1 = \nabla L^2(\theta^1)$ L^2

$$\text{update } \theta^2 \leftarrow \theta^1 - \eta g^1$$

➤ Compute gradient $g^2 = \nabla L^3(\theta^2)$ L^3

$$\text{update } \theta^3 \leftarrow \theta^2 - \eta g^2$$

1 epoch = see all the batches once → Shuffle after each epoch



Training data: $\{(x^1, \hat{y}^1), (x^2, \hat{y}^2), \dots, (x^N, \hat{y}^N)\}$

Testing data: $\{x^{N+1}, x^{N+2}, \dots, x^{N+M}\}$

把所有样本分为一个一个Batch，大小为B，计算batch内样本的平均梯度，并更新参数

所有样本计算一遍，称为一个epoch

每个epoch开始之前分一次batch，每次不同，称为shuffle

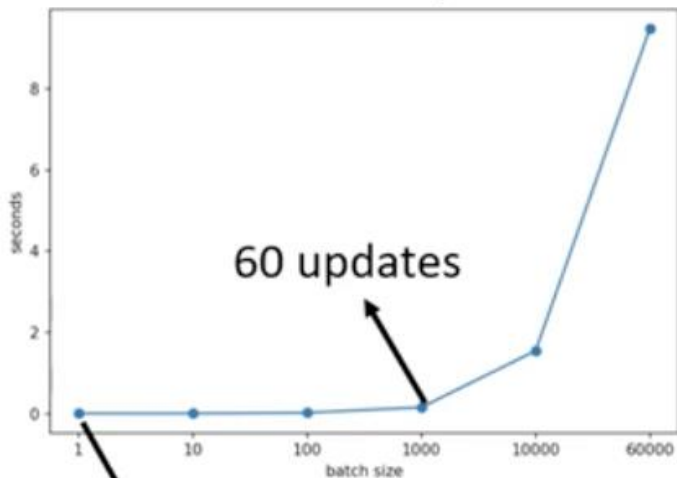


Batch大小设置

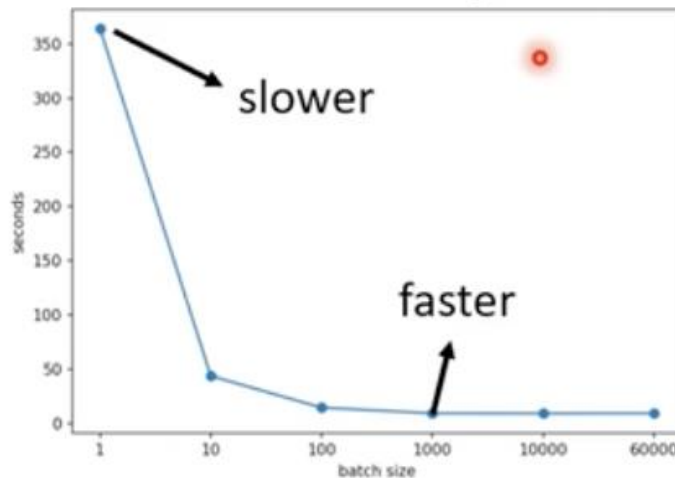
大batch不一定慢，考虑并行计算；小batch需要更多时间完成一次epoch，需要更长时间遍历一次所有样本

MINIST数字分类 Tesla V100 GPU

Time for one update



Time for one epoch



Batch大小
设为1000比
1，遍历所
有样本的时
间更快





Batch大小设置

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

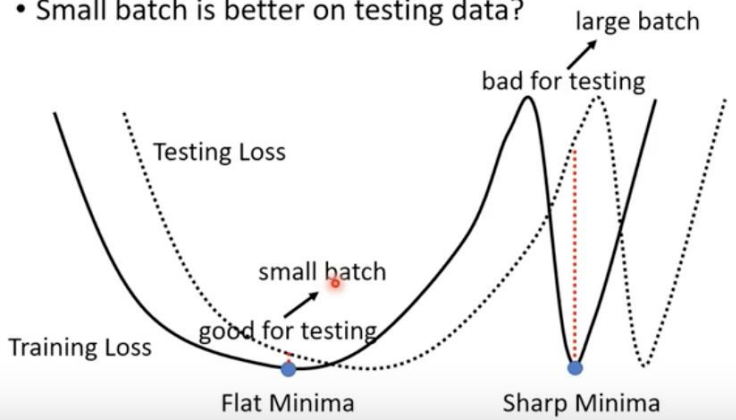
<https://arxiv.org/abs/1609.04836>

Name	Training Accuracy		Testing Accuracy	
	SB	LB	SB	LB
F_1	99.66% $\pm$ 0.05%	99.92% $\pm$ 0.01%	98.03% $\pm$ 0.07%	97.81% $\pm$ 0.07%
F_2	99.99% $\pm$ 0.03%	98.35% $\pm$ 2.08%	64.02% $\pm$ 0.2%	59.45% $\pm$ 1.05%
C_1	99.89% $\pm$ 0.02%	99.66% $\pm$ 0.2%	80.04% $\pm$ 0.12%	77.26% $\pm$ 0.42%
C_2	99.99% $\pm$ 0.04%	99.99% $\pm$ 0.01%	89.24% $\pm$ 0.12%	87.26% $\pm$ 0.07%
C_3	99.56% $\pm$ 0.44%	99.88% $\pm$ 0.30%	49.58% $\pm$ 0.39%	46.45% $\pm$ 0.43%
C_4	99.10% $\pm$ 1.23%	99.57% $\pm$ 1.84%	63.08% $\pm$ 0.5%	57.81% $\pm$ 0.17%

大Batch在测试集上表现不佳

Batch大小是需要调的参数

- Small batch is better on testing data?

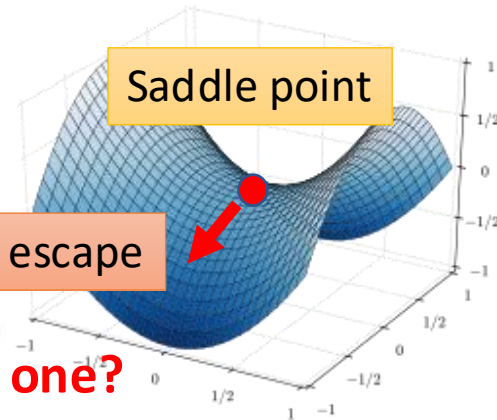
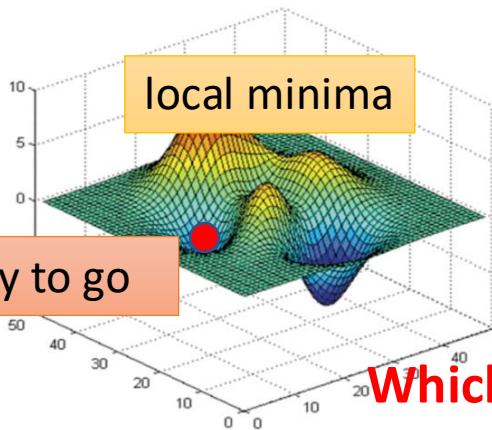
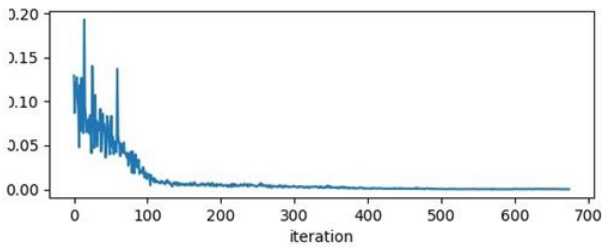


	Small	Large
Speed for one update (no parallel)	Faster	Slower
Speed for one update (with parallel)	Same	Same (not too large)
Time for one epoch	Slower	Faster
Gradient	Noisy	Stable
Optimization	Better	Worse
Generalization	Better	Worse





梯度问题



training
loss

No way to go

escape

Which one?

gradient is close
to zero

critical
point

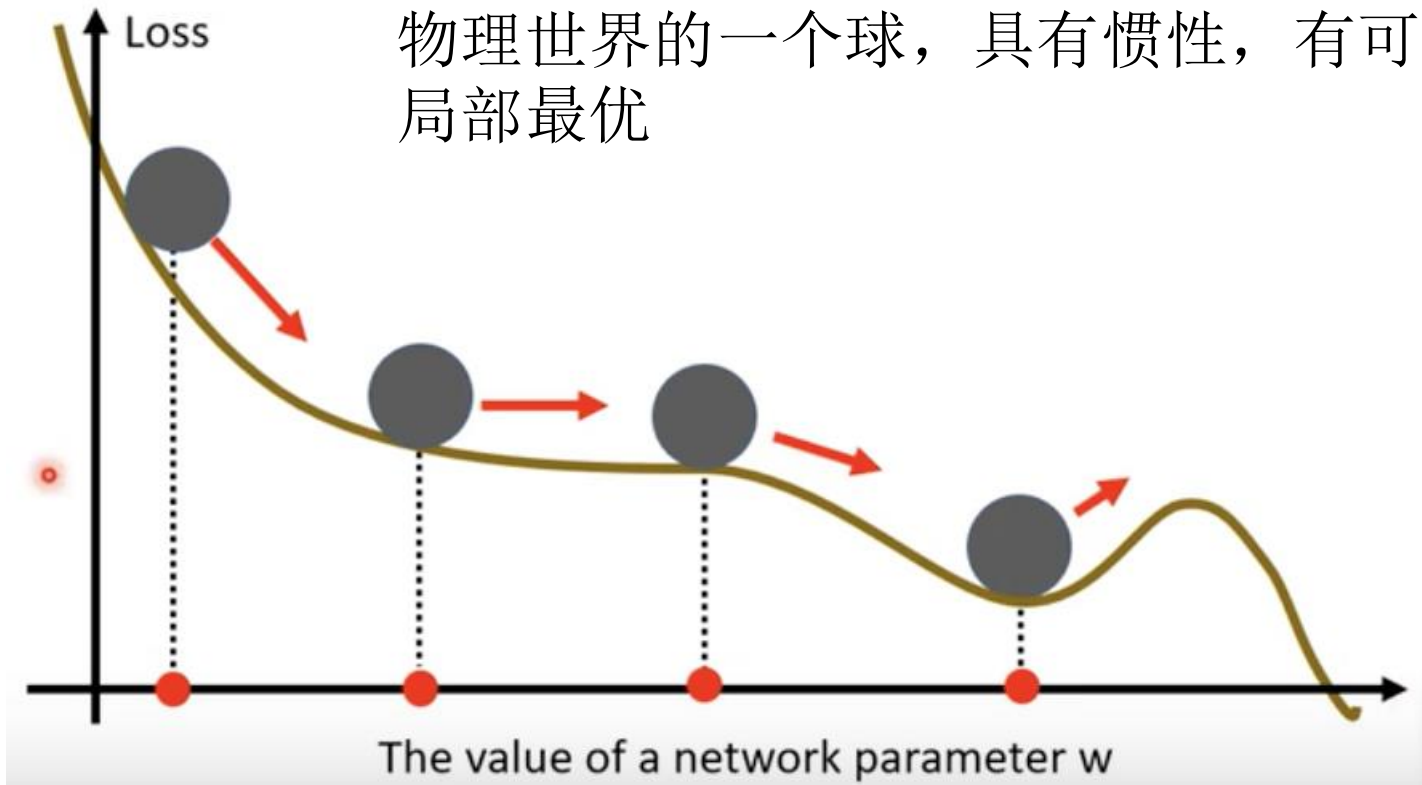
updates

Not small
enough





物理世界的一个球，具有惯性，有可能越过局部最优

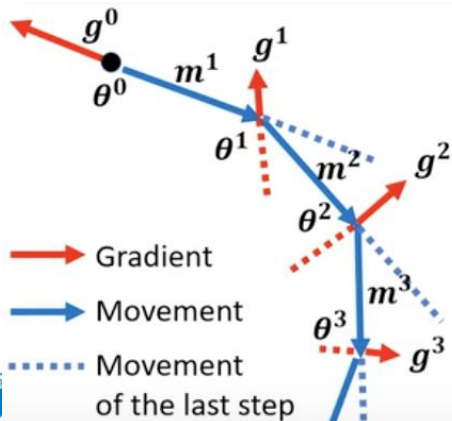




一般的梯度下降 (Vanilla SGD) $\Rightarrow$

参数更新时, 考虑惯性, 加上前一步的方向

动量: 上一次移动的方向-当前梯度



Starting at θ^0

Movement $m^0 = 0$

Compute gradient g^0

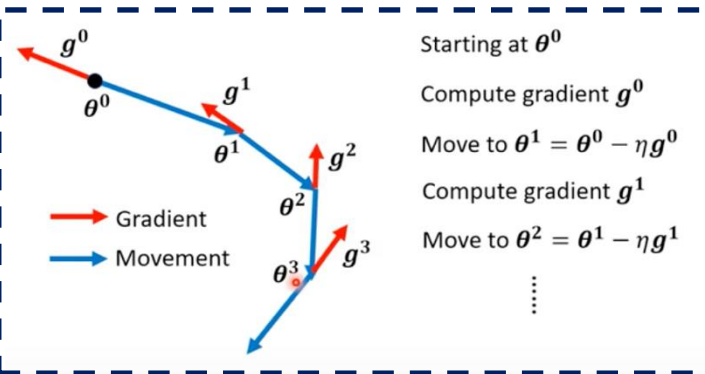
Movement $m^1 = \lambda m^0 - \eta g^0$

Move to $\theta^1 = \theta^0 + m^1$

Compute gradient g^1

Movement $m^2 = \lambda m^1 - \eta g^1$

Move to $\theta^2 = \theta^1 + m^2$



m^i is the weighted sum of all the previous gradient: $g^0, g^1, \dots, g^{i-1}$

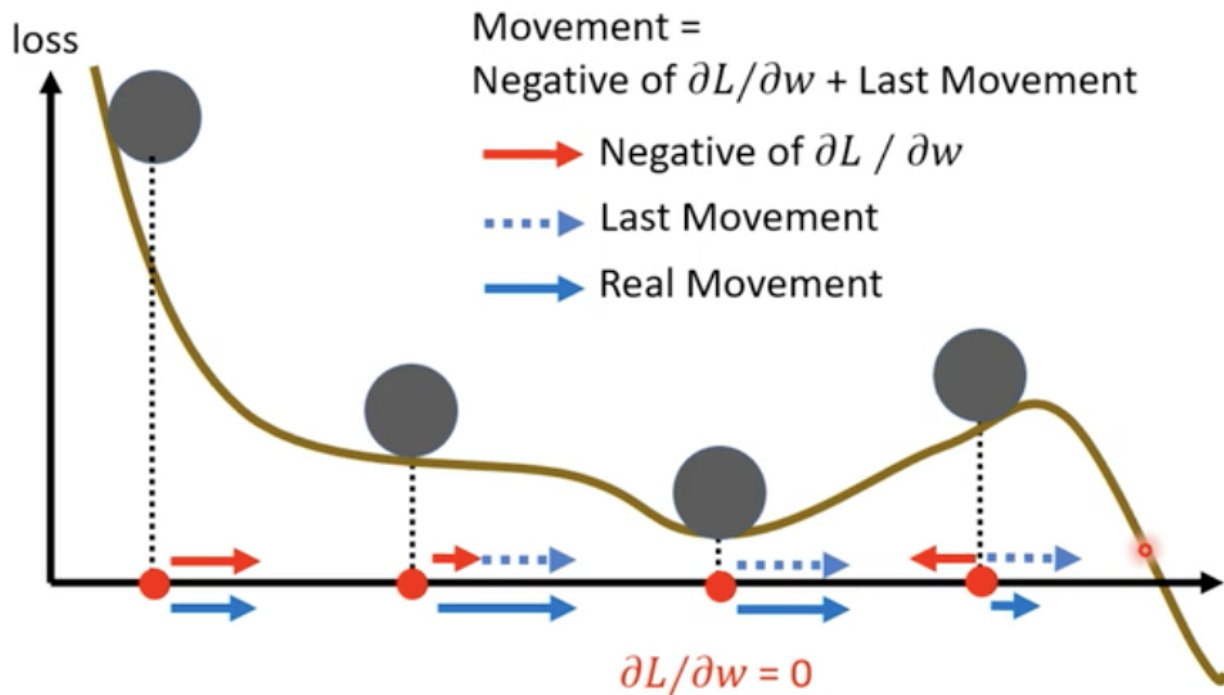
$$m^0 = 0$$

$$m^1 = -\eta g^0$$

$$m^2 = -\lambda \eta g^0 - \eta g^1$$

...

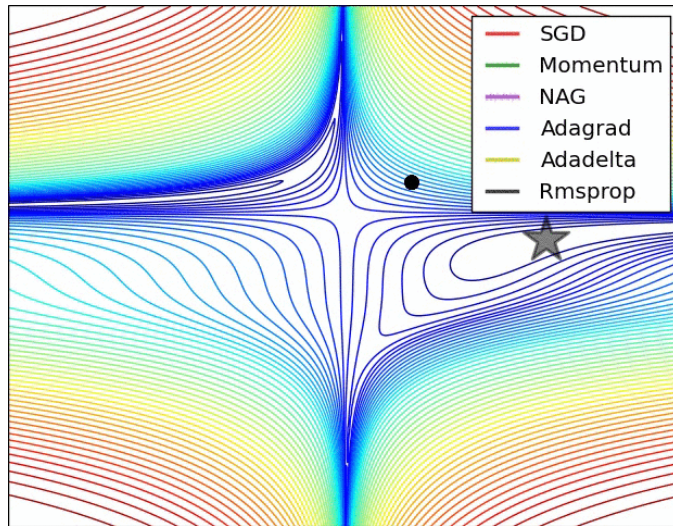
考虑过去所有方向的和



动量帮助小球在局部最优点继续向前



- **NAG (Nesterov accelerated gradient)**: 加入二阶动量, 模拟下一步参数更新后的值, 然后将模拟后的位置处梯度替换动量方法中的当前位置梯度
- **Adagrad**: 引入了二阶动量, 让学习率根据学习数据的特征自动调整其大小
- **Adadelat**: 修改二阶动量
- **RMSprop**: Hinton提出, 与Adadelat类似, γ 设置为0.9, 学习率可为固定值为0.001
- **Adam**: 自适应矩估计, **自适应学习率算法**, 将动量和Adadelat或RMSprop结合, 引入两个参数 β_1 和 β_2
- **AdaMax**: 扩展到p阶动量



更多优化器出现: Nadam、AMSGrad....

4.2. Implementation Details

We train and evaluate our method on a single server with an NVIDIA RTX 3090 GPU. The network is implemented within PyTorch and trained using AdamW [26] with a batch size of 32. We adopt the hand center provided by [31] to crop the hand from the original depth image. We resize the cropped image to a fixed size of 128×128 . The depth values are normalized to $[-1, 1]$ for the cropped image. To generate synthetic data, we randomly sample 200K hand pose data from the BigHand2.2M dataset [64]. Then, we use an iterative optimization method [1] to obtain the MANO parameters and render the 3D hand mesh obtained from the MANO as the synthetic depth image. More details are provided in the supplementary material.

Implementation For natural language, we use 300d Glove [38] vectors as word embeddings. For the words not in the vocabulary of Glove, we generate their embeddings randomly. For video modality, we use 1024d I3D feature pre-trained on Kinetics dataset [2] or 500d C3D feature [41] pre-trained on Sports-1M dataset [20] as the initial frame features, and downsample the videos at a frame rate of 1 frame per second. For each training epoch, we will re-generate a pseudo video for each video-query pair. We train the model with a batch size of 32 for 30 epochs for all datasets using Adam optimizer with an initial learning rate of 0.001. We set λ_1 , λ_2 and λ_3 in Eq. 12 to 1, 1, 1, respectively. More details are provided in supplementary material.

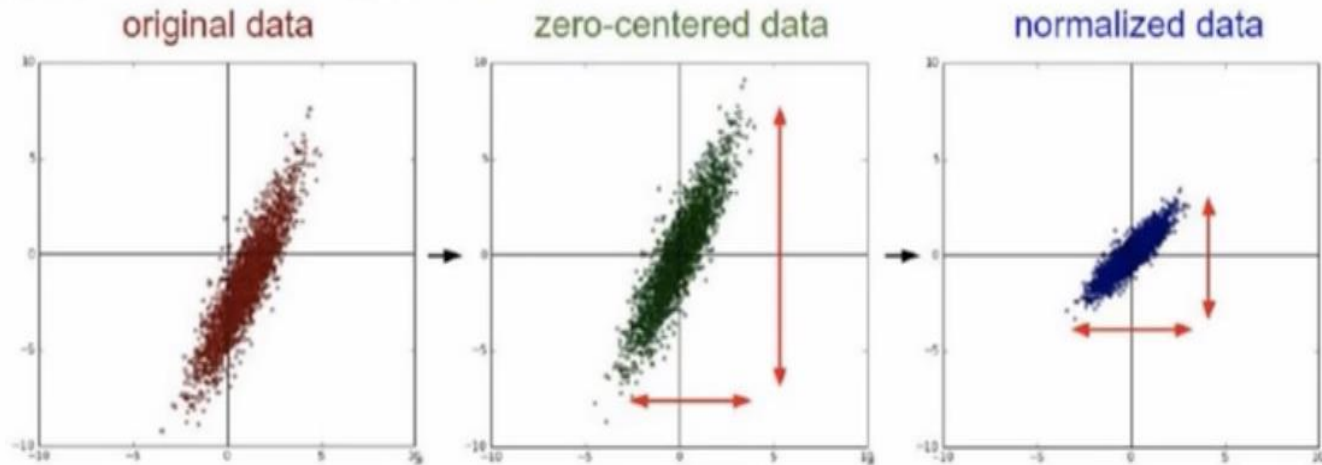
Can Shuffling Video Benefit Temporal Bias Problem: A Novel Training Framework for Temporal Grounding,
ECCV 2022

Mining Multi-View Information: A Strong Self-Supervised Framework for Depth-based 3D Hand Pose and Mesh Estimation, CVPR 2022



初始数据归一化

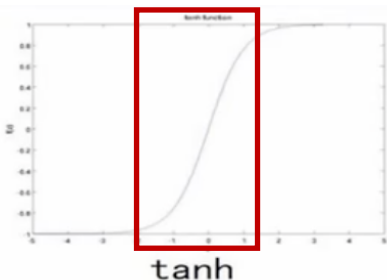
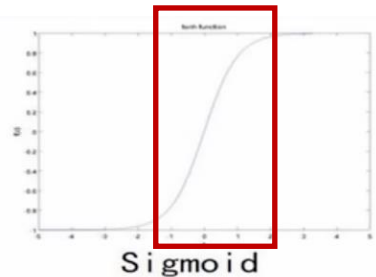
建议：做均值和方差归一化。。



每个维度做归一化，使其对结果的影响近似



初始数据归一化



很像线性函数了，需要将一部分点放在这个之外

梯度消失现象：如果 $W^T X + b$ 一开始很大或很小，那么梯度将趋近于0，反向传播后前面与之相关的梯度也趋近于0，导致训练缓慢。

因此，我们要使 $W^T X + b$ 一开始在零附近。

一种比较简单有效的方法是：(W, b) 初始化从区间 $\left(-\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}}\right)$ 均匀随机取值。其中 d 为 (W, b) 所在层的神经元个数。

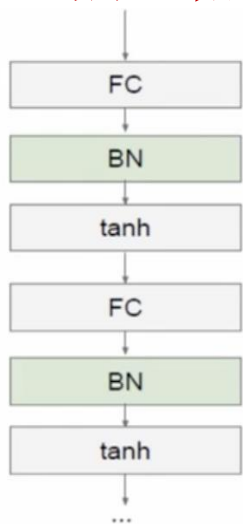
可以证明，如果 X 服从正态分布，均值0，方差1，且各个维度无关，而 (W, b) 是 $\left(-\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}}\right)$ 的均匀分布，则 $W^T X + b$ 是均值为0，方差为1/3的正态分布。





Batch Normalization

- (Google 2015) 希望每层获得的值（下一层的输入）都在0附近，从而避免梯度消失现象，**可以将每层值直接做基于均值和方差的归一化，大多数点落在0附近**



每一层FC (Fully Connected Layer)
接一个BN (Batch Normalization) 层

每层输出结果减去均值，除以方差

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1, \dots, x_m\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

将结果限制在0附近，导致失去非线性，要再次放缩
 γ 和 β 也是参数，求偏导

Batch normalization accelerating deep network training by reducing internal covariate shift, 2015



目标函数也称为损失函数，即loss，可以有多种形式。

可加正则项 (**Regulation Term**)

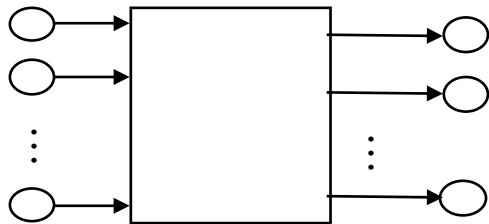
$$\begin{aligned} L(w) &= F(w) + R(w) \\ &= \frac{1}{2} \left(\sum_{i=1}^{batch_size} \|y_i - Y_i\|^2 + \beta \sum_k \sum_l w_{k,l}^2 \right) \end{aligned}$$

最小化损失函数的同时，不能让 w 很大

有时候效果会好

如果是分类问题， $F(w)$ 可以采用**SOFTMAX**函数和交叉熵的组合

SOFTMAX函数

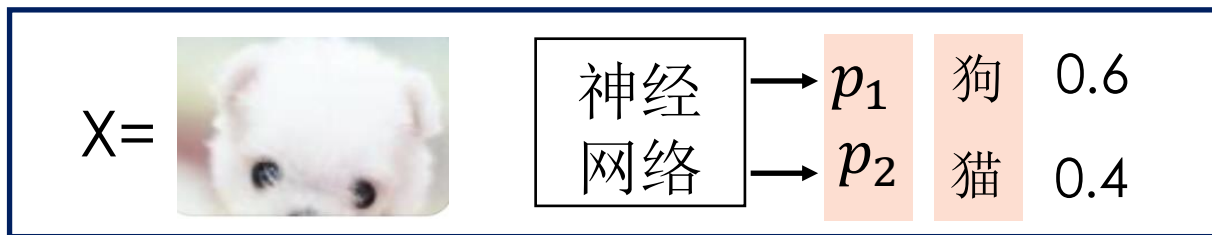


$$q_i = \frac{\exp(z_i)}{\sum_{j=1}^N \exp(z_j)}$$

转换为概率

我们希望通过此网络学习由 $Z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{bmatrix}$ 到 $P = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_N \end{bmatrix}$ 的映射，其中 $\sum_{i=1}^N p_i = 1$

样本中各类别的分布





目标函数

均方误差MSE (Mean score error): $E = \frac{1}{2} \|q - p\|^2$

交叉熵函数 (Cross Entropy)

$$\underbrace{E = - \sum_{i=1}^N p_i * \log(q_i)}_{>0}$$

交叉熵是信息论中的一个重要概念，其大小表示两个概率分布之间的差异，可以通过最小化交叉熵来得到目标概率分布的近似分布

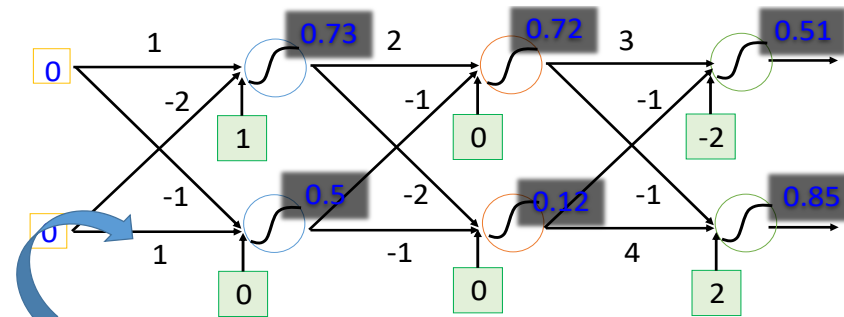
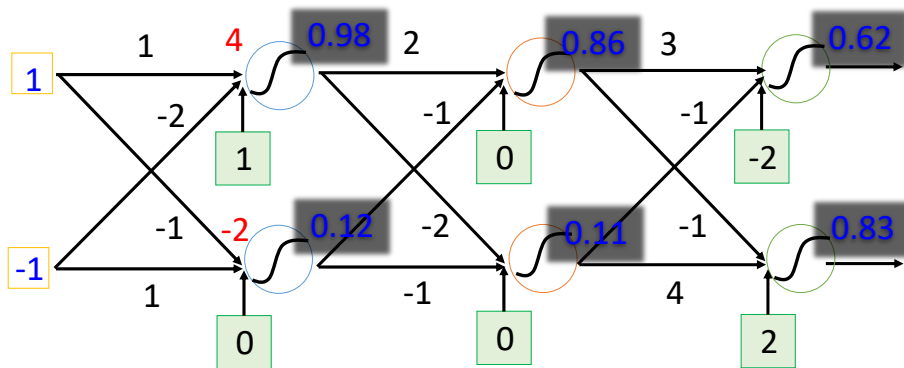
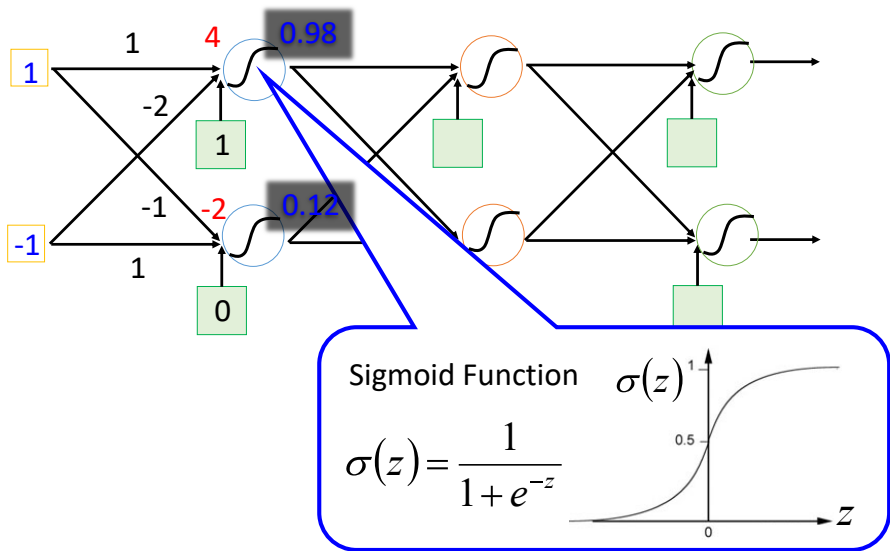
如果 $F(w)$ 可以采用SOFTMAX函数和交叉熵的组合，链式求导形式

$$\frac{\partial E}{\partial z_i} = \frac{\partial E}{\partial q_i} \cdot \frac{\partial q_i}{\partial z_i} = q_i - p_i$$

还有很多根据具体任务的目标函数的形式



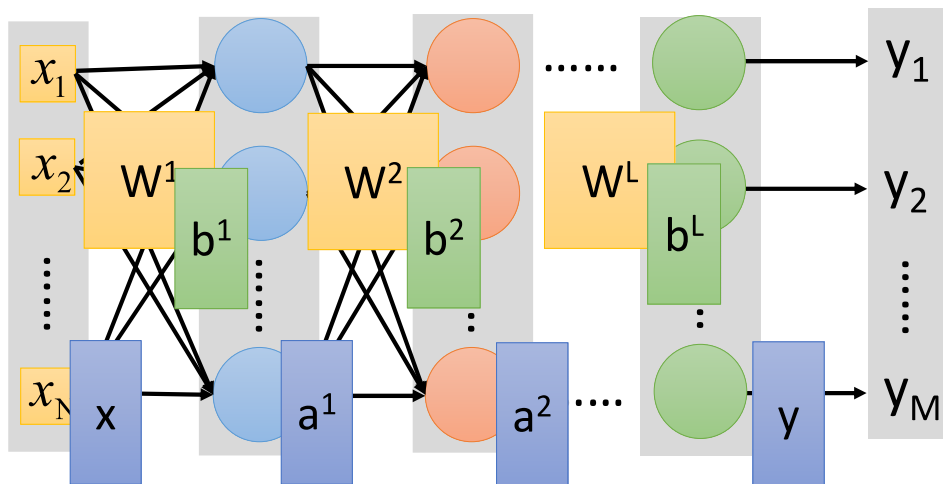
三层全连接神经网络例子



This is a function.

Input vector, output vector

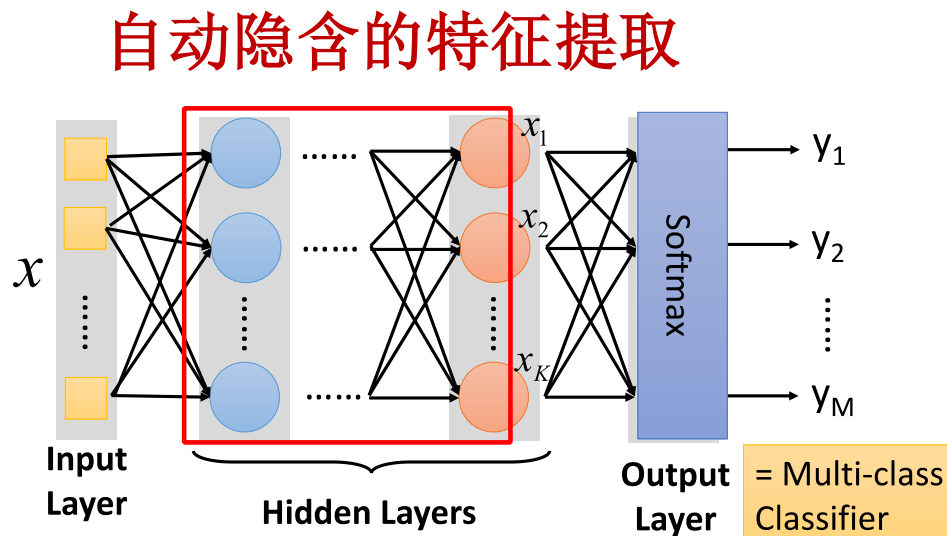
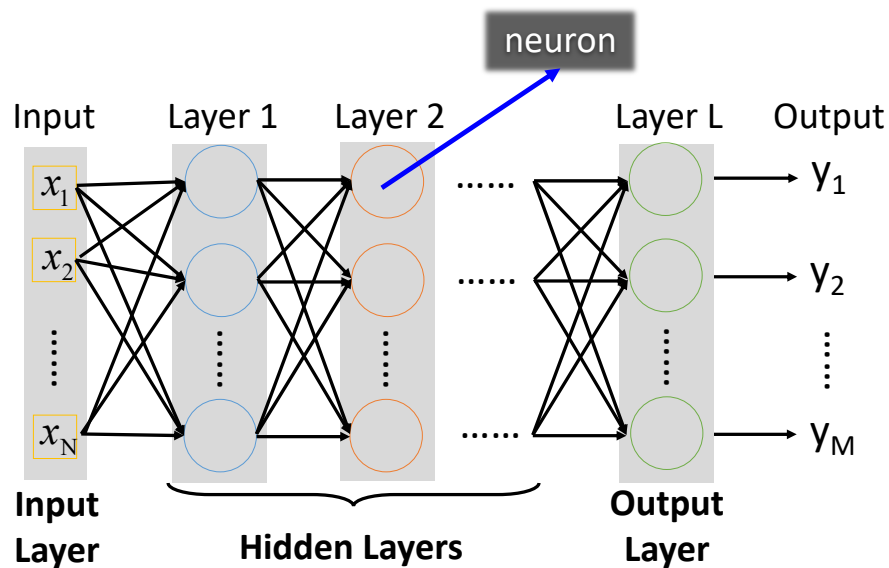
$$f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix} \quad f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.51 \\ 0.85 \end{bmatrix}$$



$$\varphi(W^1 x + b^1) \quad \varphi(W^2 a^1 + b^2) \quad \varphi(W^L a^{L-1} + b^L)$$

$$y = f(x) = \varphi(W^L \dots \varphi(W^2 \varphi(W^1 x + b^1) + b^2) \dots + b^L)$$

全连接前馈神经网络

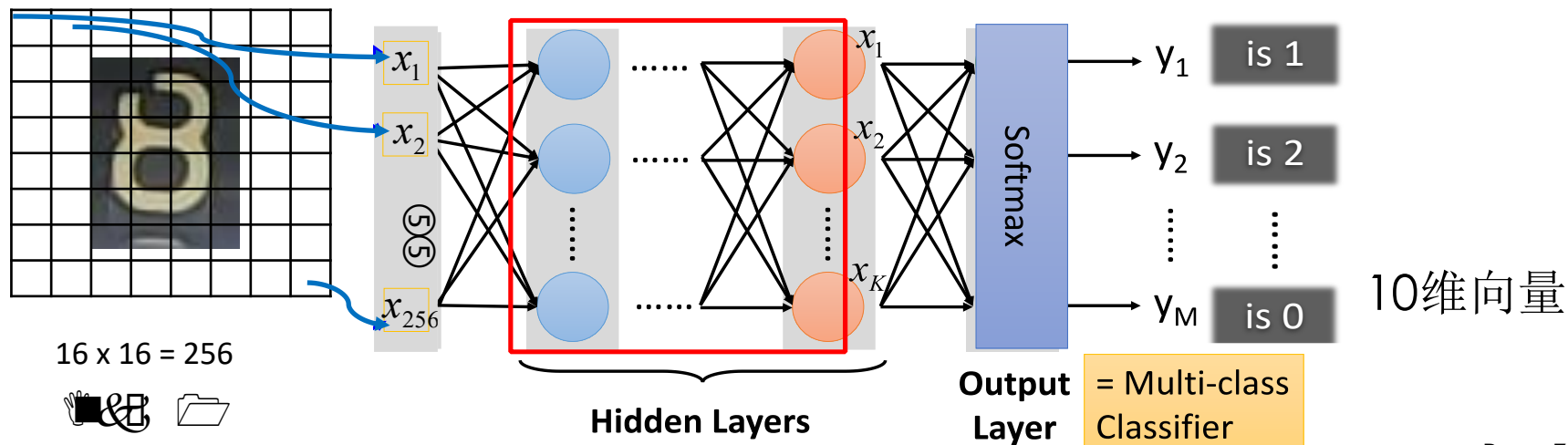


NN结构：多少个隐藏层？每层多少神经元？

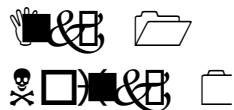


深度神经网络

“8”

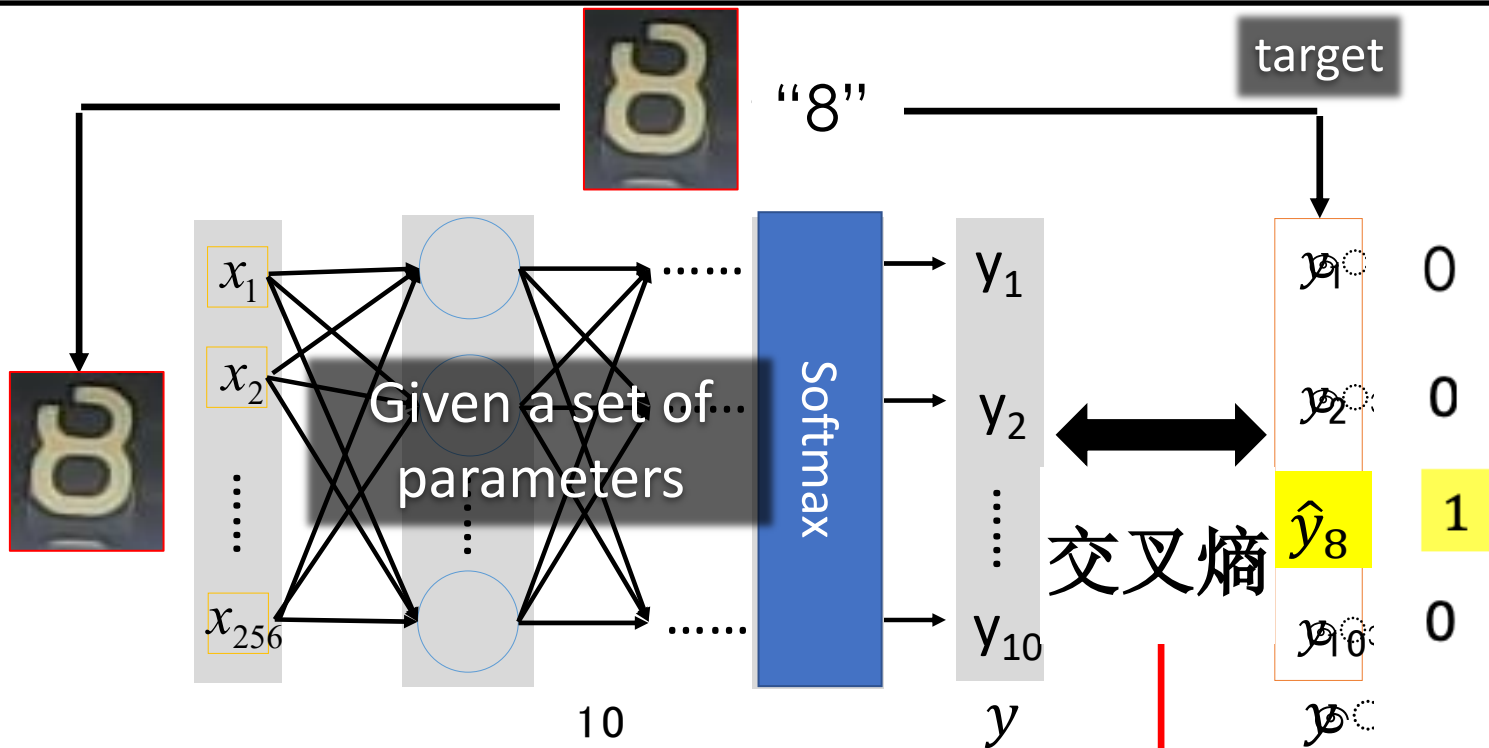


16 x 16 = 256





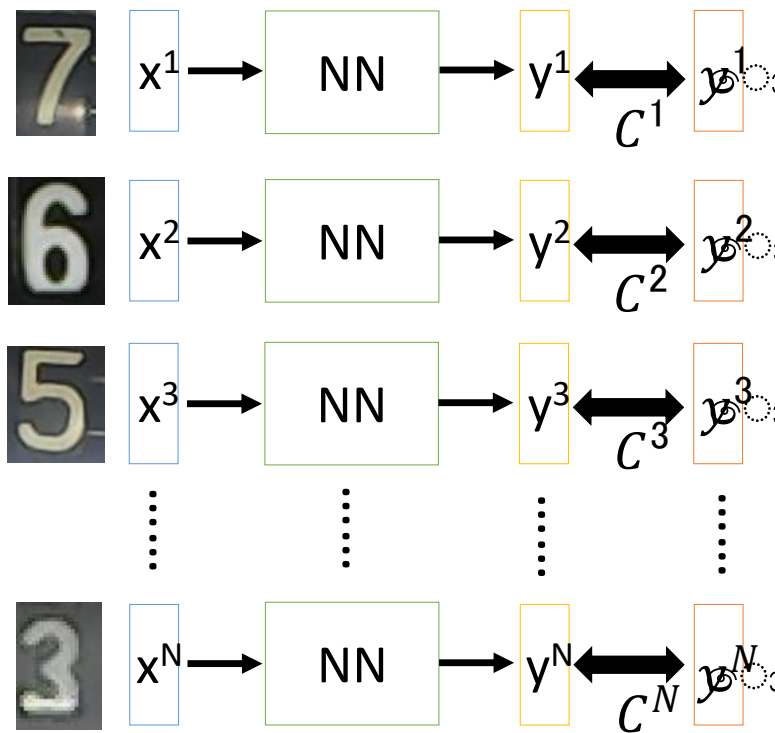
损失函数



$$C(y, p) = - \sum_{i=1}^{10} p_i \ln y_i$$



损失函数



计算所有样本
的总损失函数

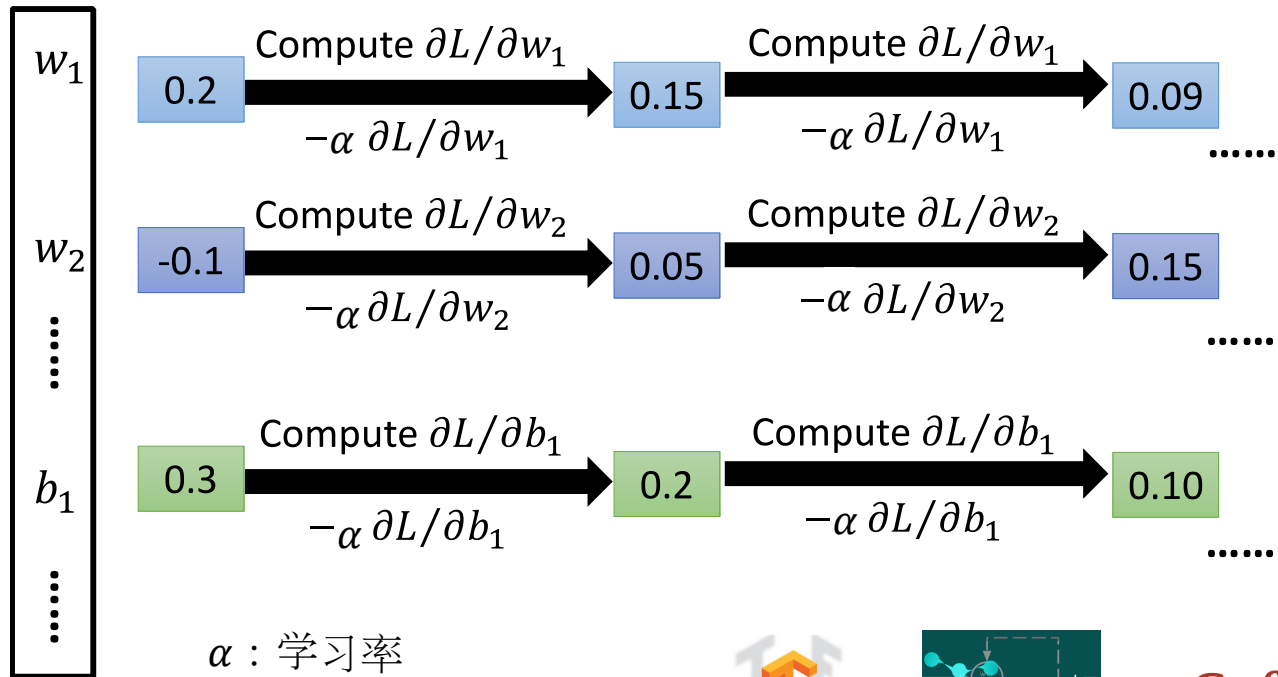
$$L = \sum_{n=1}^N C^n$$

找到最小化L的模型参数





梯度下降



反向传播计算所有参数的梯度

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

α : 学习率

可以利用深度学习框架计算



Caffe



Deeper is Better?

Layer X Size	Word Error Rate (%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1

Not surprised, more
parameters, better
performance

Seide, Frank, Gang Li, and Dong Yu. "Conversational Speech Transcription Using Context-Dependent Deep Neural Networks." *Interspeech*. 2011.